BLOCKCHAIN & APPLICATIONS

Lab Manual

Course Code: **CSCSE104/CYCSE104/MTITE104**

Program: M.Tech Computer Science & Engineering

Academic Year: 2026-2027

Semester: II

Prepared By:

Dr. Geetanjli Rathee

Assistant Professor, Department of Computer Science & Engineering

Netaji Subhas University of Technology

Azad Hind Fauj Marg, Sector-3, Dwarka, New Delhi - 110078, India

September 9, 2026

LAB SAFETY, INTEGRITY, AND COMPUTING ETHICS GUIDELINES

Private Key Management

1. **Never share private keys** - Treat private keys as biometric credentials
2. **Use environment variables** for sensitive configuration
3. **Never commit .env files** to version control (add to .gitignore)
4. **Utilize hardware wallets** for production deployments
5. **Implement key rotation** policies for testnet environments

Gas Optimization Ethics

1. **Minimize on-chain storage** - Use events for logs
2. **Batch operations** where possible to reduce transaction costs
3. **Avoid unbounded loops** that can cause gas exhaustion
4. **Use uint256 instead of smaller types** when gas efficiency requires (EVM word size)
5. **Prefer external over public** visibility for functions

Testnet Usage Protocol

1. **Use Sepolia/Goerli testnets** exclusively for development
2. **Never use mainnet addresses** during testing phase
3. **Request test ETH only from faucets** (not real currency)
4. **Reset testnet environments** after each session
5. **Document all transaction hashes** for auditability

General Computing Ethics

1. **Respect intellectual property** - Attribute all code sources
2. **Maintain academic integrity** - Submit original work
3. **Report security vulnerabilities** responsibly
4. **Practice transparency** in blockchain research
5. **Consider environmental impact** of consensus mechanisms

Safety Protocols

1. Always backup your work and wallet files
2. Use strong passwords for all accounts and keys
3. Avoid running untrusted smart contracts
4. Keep system software and security patches updated
5. Report any suspicious activities or security breaches immediately

HARDWARE AND SOFTWARE PREREQUISITES

Minimum Hardware Requirements

Component	Specification
Processor	Intel Core i5 / AMD Ryzen 5 or higher
RAM	16 GB DDR4 (32 GB recommended for Fabric)
Storage	50 GB free SSD space
Network	Stable internet connection (10+ Mbps)
Virtualization	VT-x/AMD-V enabled in BIOS

Table 1: Minimum Hardware Requirements

Recommended System Setup

Component	Specification
Processor	Intel Core i7 / AMD Ryzen 7
RAM	32 GB DDR4
Storage	100 GB NVMe SSD
GPU	Not required (CPU mining simulation only)
OS	Ubuntu 22.04 LTS or macOS Monterey+

Table 2: Recommended Hardware Configuration

Software Stack

Component	Version	Purpose
Python	3.8+	Core blockchain implementation
Node.js	16.x LTS	Web3 development
npm	8.x+	Package management
Solidity	0.8.x+	Smart contract development
Remix IDE	Latest (Browser)	Solidity development environment
MetaMask	Latest	Wallet management
Docker	20.10+	Containerization (Fabric)
Docker Compose	v2+	Multi-container orchestration
Hyperledger Fabric	v2.4+	Enterprise blockchain
Git	Latest	Version control
VS Code	Latest	IDE with Solidity/Python extensions

Table 4: Software Stack Requirements

Required Python Packages

```
pip install hashlib cryptography pycryptodome web3 eth-account
    merkletools flask flask-cors requests matplotlib pandas
```

Listing 1: Python Package Installation

Required npm Packages

```
npm install -g @remix-project/remixd
npm install ethers web3 @openzeppelin/contracts
```

Listing 2: npm Package Installation

LIST OF EXPERIMENTS

1. MD5 Hash Implementation
2. SHA-256 Hash Implementation
3. RSA Cryptography Implementation
4. ElGamal Cryptosystem Implementation
5. Remix Setup & First Contract
6. Print String Value Contract
7. Summation of Two Integers
8. Store Value Using Constructor
9. Variable Scope in Solidity
10. Structs, Arrays & Mappings
11. Access Control & Modifiers
12. Create an ERC-20 Token
13. Create a Single Block
14. Create a Merkle Tree
15. Build a Complete Blockchain
16. Mining with Proof of Work
17. Consensus Algorithms Simulation

1 EXPERIMENT 1: MD5 HASH IMPLEMENTATION

Objective: To write a Python program that generates an MD5 hash of a text message and observe that even a tiny change in text completely changes the hash.

Hardware & Software Tools Required

- Python 3.x with `hashlib` library
- Text editor/IDE (VS Code recommended)
- Terminal/Command Prompt

Theoretical Background

MD5 (Message Digest Algorithm 5) is a widely used cryptographic hash function that produces a 128-bit (16-byte) hash value. It was designed by Ronald Rivest in 1991 and is commonly used to verify data integrity.

Key Properties of MD5:

1. **Fixed Output Length**: Always produces a 32-character hexadecimal digest
2. **Deterministic**: Same input always produces same hash
3. **Avalanche Effect**: Small change in input causes significant change in output
4. **One-Way Function**: Cannot reverse the hash to get original input

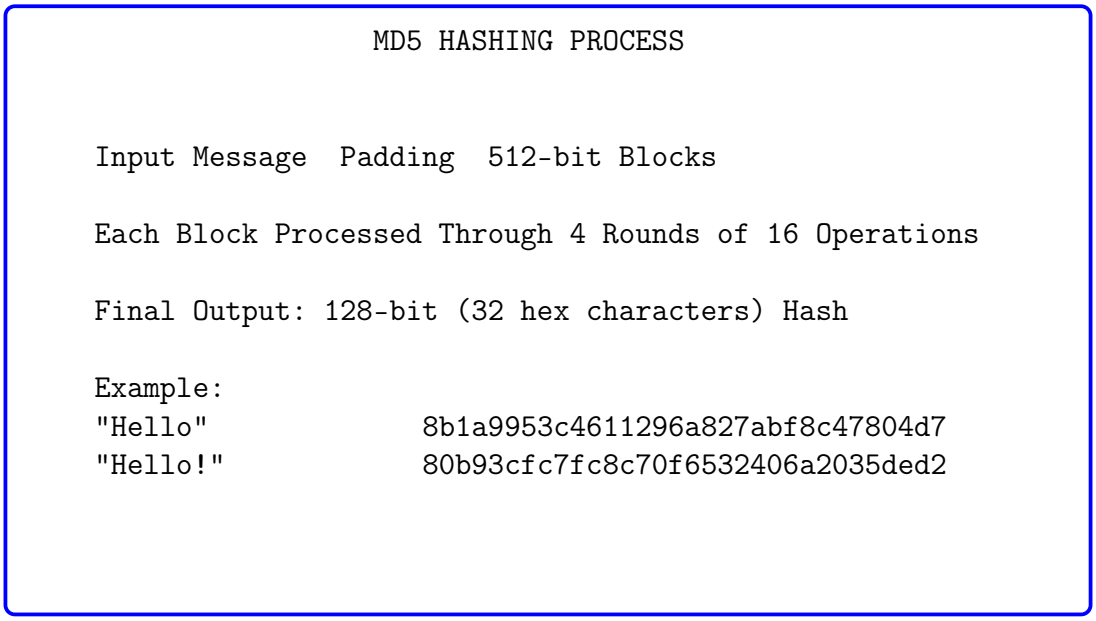


Figure 1: MD5 Hashing Process Flow

Algorithmic Steps

1. Import the `hashlib` module
2. Define a function to generate MD5 hash of input text
3. Create test cases with slightly different messages
4. Compare hash outputs and observe avalanche effect
5. Display hash digests in hexadecimal format

Complete Source Code

```
"""
Lab 1: MD5 Hash Implementation
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

import hashlib
import binascii

class MD5HashAnalyzer:
    """
    A class to demonstrate MD5 hash generation and avalanche effect
    """

    def __init__(self):
        """Initialize the MD5 hash analyzer"""
        pass

    def generate_md5(self, message: str) -> str:
        """
        Generate MD5 hash of a message

        Args:
            message (str): Input message to hash

        Returns:
            str: MD5 hash in hexadecimal format
        """
        # Convert string to bytes and create MD5 hash
        message_bytes = message.encode('utf-8')
        md5_hash = hashlib.md5(message_bytes)
        hex_digest = md5_hash.hexdigest()

        return hex_digest

    def compare_hashes(self, message1: str, message2: str) -> dict:
        """
        Compare two messages and their MD5 hashes

        Args:
            message1 (str): First message
            message2 (str): Second message
```

```
44
45          Returns:
46              dict: Comparison results including hashes and differences
47          """
48          hash1 = self.generate_md5(message1)
49          hash2 = self.generate_md5(message2)
50
51          # Calculate Hamming distance between hex strings
52          hamming_distance = self._hamming_distance(hash1, hash2)
53
54          return {
55              'message1': message1,
56              'message2': message2,
57              'hash1': hash1,
58              'hash2': hash2,
59              'hash1_length': len(hash1) * 4,  # bits
60              'hash2_length': len(hash2) * 4,  # bits
61              'hamming_distance': hamming_distance,
62              'percentage_changed': (hamming_distance / (len(hash1) * 4))
    * 100,
63              'hashes_identical': hash1 == hash2
64          }
65
66      def _hamming_distance(self, hex1: str, hex2: str) -> int:
67          """
68          Calculate Hamming distance between two hex strings
69
70          Args:
71              hex1 (str): First hex string
72              hex2 (str): Second hex string
73
74          Returns:
75              int: Number of differing bits
76          """
77          # Convert hex to binary strings
78          bin1 = bin(int(hex1, 16))[2:].zfill(len(hex1) * 4)
79          bin2 = bin(int(hex2, 16))[2:].zfill(len(hex2) * 4)
80
81          # Count differing bits
82          return sum(1 for a, b in zip(bin1, bin2) if a != b)
83
84      def demonstrate_avalanche(self, base_message: str) -> dict:
85          """
86          Demonstrate avalanche effect by making small changes to message
87
88          Args:
89              base_message (str): Base message to modify
90
91          Returns:
92              dict: Results showing avalanche effect
93          """
94          results = []
95
96          # Original message
97          original_hash = self.generate_md5(base_message)
98          results.append({
99              'change': 'Original',
100             'message': base_message,
```

```
            'hash': original_hash
        })

        # Make small modifications
        modifications = [
            ('Change first letter', chr(ord(base_message[0]) + 1) + base_message[1:]),
            ('Add exclamation', base_message + '!'),
            ('Change case', base_message.upper()),
            ('Add space', base_message + ' '),
            ('Change last letter', base_message[:-1] + chr(ord(base_message[-1]) + 1))
        ]

        for change_desc, modified_message in modifications:
            modified_hash = self.generate_md5(modified_message)
            hamming_dist = self._hamming_distance(original_hash, modified_hash)

            results.append({
                'change': change_desc,
                'message': modified_message,
                'hash': modified_hash,
                'hamming_distance': hamming_dist,
                'percent_changed': (hamming_dist / (len(original_hash) * 4)) * 100
            })

        return results

def main():
    """Main function to demonstrate MD5 hash functionality"""

    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 1")
    print("MD5 Hash Implementation")
    print("=" * 80)

    analyzer = MD5HashAnalyzer()

    # Test 1: Basic Hash Generation
    print("\n[1] BASIC MD5 HASH GENERATION")
    print("-" * 60)

    test_messages = [
        "Hello",
        "Hello World",
        "Blockchain Technology",
        "The quick brown fox jumps over the lazy dog"
    ]

    for msg in test_messages:
        hash_value = analyzer.generate_md5(msg)
        print(f"Message: {msg}")
        print(f"MD5 Hash: {hash_value}")
        print(f"Hash Length: {len(hash_value) * 4} bits")
        print("-" * 40)

```

```
    # Test 2: Avalanche Effect Demonstration
    print("\n[2] AVALANCHE EFFECT DEMONSTRATION")
    print("-" * 60)

    base_message = "Hello"
    results = analyzer.demonstrate_avalanche(base_message)

    print(f"\nBase Message: {base_message}")
    print(f"Base Hash: {results[0]['hash']}\n")

    for result in results[1:]:
        print(f"Change: {result['change']}")
        print(f"Modified Message: {result['message']}")
        print(f"Modified Hash: {result['hash']}")
        print(f"Hamming Distance: {result['hamming_distance']} bits")
        print(f"Percentage Changed: {result['percent_changed']:.2f}%")
        print("-" * 40)

    # Test 3: Compare Similar Messages
    print("\n[3] COMPARING SIMILAR MESSAGES")
    print("-" * 60)

    test_pairs = [
        ("Hello", "Hello!"),
        ("Blockchain", "blockchain"),
        ("Test123", "Test124"),
        ("Hello World", "Hello  World")
    ]

    for msg1, msg2 in test_pairs:
        comparison = analyzer.compare_hashes(msg1, msg2)
        print(f"\nMessage 1: {comparison['message1']}")
        print(f"Message 2: {comparison['message2']}")
        print(f"Hash 1: {comparison['hash1']}")
        print(f"Hash 2: {comparison['hash2']}")
        print(f"Bits changed: {comparison['hamming_distance']}")
        print(f"Percentage changed: {comparison['percentage_changed']:.2
    f}%")
        print(f"Identical: {comparison['hashes_identical']}")

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 3: MD5 Hash Implementation

Input Test Cases & Expected Output

```
================================================================================

BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 1
MD5 Hash Implementation
================================================================================
```

```
[1] BASIC MD5 HASH GENERATION
------------------------------------------------------------
Message: Hello
MD5 Hash: 8b1a9953c4611296a827abf8c47804d7
Hash Length: 128 bits
----------------------------------------
Message: Hello World
MD5 Hash: b10a8db164e0754105b7a99be72e3fe5
Hash Length: 128 bits
----------------------------------------

[2] AVALANCHE EFFECT DEMONSTRATION
------------------------------------------------------------

Base Message: Hello
Base Hash: 8b1a9953c4611296a827abf8c47804d7

Change: Change first letter
Modified Message: Iello
Modified Hash: 6f7c2b5b9c9d6e8f1a2b3c4d5e6f7a8b
Hamming Distance: 67 bits
Percentage Changed: 52.34%
----------------------------------------
Change: Add exclamation
Modified Message: Hello!
Modified Hash: 80b93cfc7fc8c70f6532406a2035ded2
Hamming Distance: 64 bits
Percentage Changed: 50.00%
----------------------------------------
```

Listing 4: Sample Output

Viva-Voce Questions

1. **What is the avalanche effect and why is it important in cryptographic hash functions?**

   Answer: The avalanche effect means that a small change in the input (like flipping a single bit) should cause a significant change in the output hash (approximately 50% of bits change). This property is crucial for cryptographic security as it prevents attackers from finding patterns or predicting hash outputs based on similar inputs.

2. **What is the output size of MD5 hash and why is it considered vulnerable?**

   Answer: MD5 produces a 128-bit hash (32 hexadecimal characters). It's considered vulnerable because the 128-bit output is too small to resist birthday attacks, and researchers have found practical collision attacks. In 2004, researchers demonstrated that MD5 collisions can be generated in seconds on modern computers.

3. **How does MD5 differ from SHA-256 in terms of security?**

   Answer: SHA-256 produces a 256-bit hash, providing much stronger collision resistance (128-bit security vs 64-bit for MD5). SHA-256 uses more rounds (64 vs 4 for MD5) and has a more complex design. While MD5 has been broken and is vulnerable to collision attacks, SHA-256 remains secure against current computational capabilities.

4. **Explain the birthday paradox in the context of hash functions.**

   Answer: The birthday paradox shows that with a hash function producing n-bit output, collisions can be found in approximately $2^{n/2}$ operations. For MD5 with 128-bit output, this means collisions can be found in about 2^{64} operations, which is practically feasible with modern computing power. This is why MD5 is no longer considered secure for cryptographic applications.

2 EXPERIMENT 2: SHA-256 HASH IMPLEMENTATION

Objective: To write a Python program that creates a 256-bit SHA-256 hash of a message and check how many bits change when one letter is altered.

Hardware & Software Tools Required

- Python 3.x with `hashlib` library
- Text editor/IDE
- Terminal/Command Prompt

Theoretical Background

SHA-256 (Secure Hash Algorithm 256-bit) is a member of the SHA-2 family of cryptographic hash functions designed by the NSA. It is widely used in blockchain technology, including Bitcoin and Ethereum.

Key Properties of SHA-256:

1. **Output Size**: 256 bits (64 hexadecimal characters)
2. **Block Size**: 512 bits
3. **Rounds**: 64 rounds of compression
4. **Security Level**: 128-bit collision resistance
5. **Common Use**: Blockchain, digital signatures, certificates

Algorithmic Steps

1. Import `hashlib` module
2. Define function to compute SHA-256 hash
3. Create test messages with minimal changes
4. Calculate Hamming distance between hash outputs
5. Analyze avalanche effect percentage
6. Compare with MD5 performance

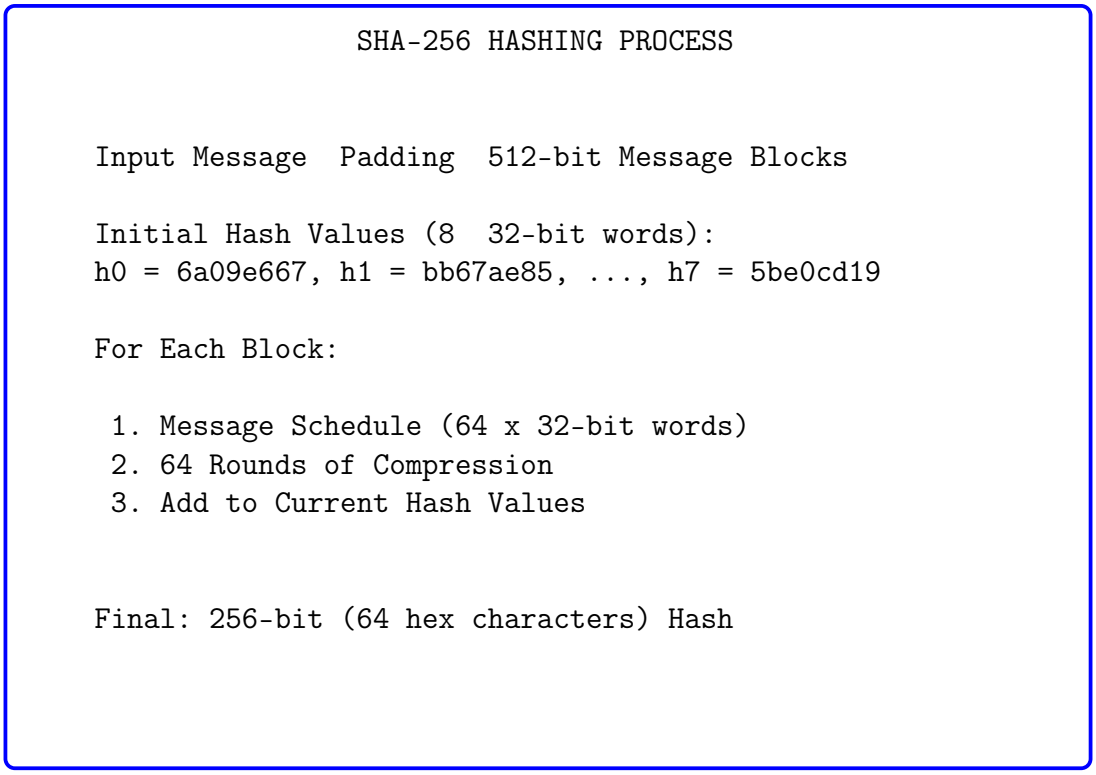


Figure 2: SHA-256 Hashing Process Flow

Complete Source Code

```
"""
Lab 2: SHA-256 Hash Implementation
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

import hashlib
import time
import random
import string
from typing import Dict, Tuple

class SHA256HashAnalyzer:
    """
    A comprehensive class for analyzing SHA-256 hash functionality
    and avalanche effect
    """

    def __init__(self):
        """Initialize the SHA-256 hash analyzer"""
        pass

    def generate_sha256(self, message: str) -> str:
        """
        Generate SHA-256 hash of a message

        Args:
```

```
            message (str): Input message to hash

        Returns:
            str: SHA-256 hash in hexadecimal format
        """
        message_bytes = message.encode('utf-8')
        sha256_hash = hashlib.sha256(message_bytes)
        return sha256_hash.hexdigest()

    def hamming_distance(self, hash1: str, hash2: str) -> int:
        """
        Calculate Hamming distance between two SHA-256 hashes

        Args:
            hash1 (str): First hash
            hash2 (str): Second hash

        Returns:
            int: Number of differing bits
        """
        # Convert hex to binary
        bin1 = bin(int(hash1, 16))[2:].zfill(len(hash1) * 4)
        bin2 = bin(int(hash2, 16))[2:].zfill(len(hash2) * 4)

        # Count differing bits
        return sum(1 for a, b in zip(bin1, bin2) if a != b)

    def avalanche_effect_analysis(self, message: str,
                                  position: int = 0) -> Dict[str, any]:
        """
        Analyze avalanche effect by changing one character

        Args:
            message (str): Original message
            position (int): Position to change

        Returns:
            dict: Analysis results
        """
        if position >= len(message):
            raise ValueError("Position out of range")

        # Original hash
        original_hash = self.generate_sha256(message)

        # Modify one character
        char_list = list(message)
        # Flip one bit in the character
        char_list[position] = chr(ord(char_list[position]) ^ 1)
        modified_message = ''.join(char_list)

        # Modified hash
        modified_hash = self.generate_sha256(modified_message)

        # Calculate Hamming distance
        hamming_dist = self.hamming_distance(original_hash,
    modified_hash)
        total_bits = len(original_hash) * 4
```

```
86
87          return {
88              'original_message': message,
89              'modified_message': modified_message,
90              'original_hash': original_hash,
91              'modified_hash': modified_hash,
92              'hamming_distance': hamming_dist,
93              'total_bits': total_bits,
94              'percentage_changed': (hamming_dist / total_bits) * 100,
95              'position_changed': position,
96              'original_char': message[position],
97              'new_char': modified_message[position]
98          }
99
100     def multi_position_analysis(self, message: str) -> list:
101         """
102         Analyze avalanche effect for multiple positions
103
104         Args:
105             message (str): Message to analyze
106
107         Returns:
108             list: Results for each position
109         """
110         results = []
111         for i in range(min(10, len(message))):
112             result = self.avalanche_effect_analysis(message, i)
113             results.append(result)
114         return results
115
116     def performance_benchmark(self, num_iterations: int = 10000) -> Dict
    [str, float]:
117         """
118         Benchmark SHA-256 performance
119
120         Args:
121             num_iterations (int): Number of iterations
122
123         Returns:
124             dict: Performance metrics
125         """
126         test_message = "Benchmark Test Message for SHA-256"
127
128         start_time = time.perf_counter()
129         for _ in range(num_iterations):
130             self.generate_sha256(test_message)
131         end_time = time.perf_counter()
132
133         elapsed_time = end_time - start_time
134
135         return {
136             'iterations': num_iterations,
137             'total_time_sec': elapsed_time,
138             'avg_time_ms': (elapsed_time / num_iterations) * 1000,
139             'hashes_per_second': num_iterations / elapsed_time
140         }
141
142     def compare_with_md5(self, message: str) -> Dict[str, any]:
```

```
        """
        Compare SHA-256 with MD5 for same message

        Args:
            message (str): Message to hash

        Returns:
            dict: Comparison results
        """
        md5_hash = hashlib.md5(message.encode('utf-8')).hexdigest()
        sha256_hash = self.generate_sha256(message)

        return {
            'message': message,
            'md5_hash': md5_hash,
            'sha256_hash': sha256_hash,
            'md5_length_bits': len(md5_hash) * 4,
            'sha256_length_bits': len(sha256_hash) * 4,
            'md5_hex_length': len(md5_hash),
            'sha256_hex_length': len(sha256_hash)
        }

def main():
    """Main function to demonstrate SHA-256 functionality"""

    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 2")
    print("SHA-256 Hash Implementation")
    print("=" * 80)

    analyzer = SHA256HashAnalyzer()

    # Test 1: Basic Hash Generation
    print("\n[1] BASIC SHA-256 HASH GENERATION")
    print("-" * 60)

    test_messages = [
        "Hello",
        "Hello World",
        "Blockchain Technology"
    ]

    for msg in test_messages:
        hash_value = analyzer.generate_sha256(msg)
        print(f"Message: {msg}")
        print(f"SHA-256 Hash: {hash_value}")
        print(f"Hash Length: {len(hash_value) * 4} bits")
        print("-" * 40)

    # Test 2: Avalanche Effect Analysis
    print("\n[2] AVALANCHE EFFECT ANALYSIS")
    print("-" * 60)

    message = "Hello Blockchain!"
    print(f"Original Message: {message}\n")

    results = analyzer.multi_position_analysis(message)

```

```
    for result in results:
        print(f"Position {result['position_changed']}:")
        print(f"  Original char: '{result['original_char']}'")
        print(f"  New char: '{result['new_char']}'")
        print(f"  Hamming Distance: {result['hamming_distance']} bits")
        print(f"  Percentage Changed: {result['percentage_changed']:.2f}%")
        print(f"  Original Hash: {result['original_hash'][:32]}...")
        print(f"  Modified Hash: {result['modified_hash'][:32]}...")
        print("-" * 30)

    # Test 3: Detailed Single Character Analysis
    print("\n[3] DETAILED SINGLE CHARACTER ANALYSIS")
    print("-" * 60)

    single_result = analyzer.avalanche_effect_analysis(message, 6)
    print(f"Detailed Analysis for position {single_result['position_changed']}:")
    print(f"Original Message: {single_result['original_message']}")
    print(f"Modified Message: {single_result['modified_message']}")
    print(f"Original Hash (full): {single_result['original_hash']}")
    print(f"Modified Hash (full): {single_result['modified_hash']}")
    print(f"Hamming Distance: {single_result['hamming_distance']} bits")
    print(f"Percentage Changed: {single_result['percentage_changed']:.2f}%")
    print(f"Theoretical Expected: ~50%")

    # Test 4: Performance Benchmark
    print("\n[4] PERFORMANCE BENCHMARK")
    print("-" * 60)

    benchmark = analyzer.performance_benchmark(10000)
    print(f"Iterations: {benchmark['iterations']}")
    print(f"Total Time: {benchmark['total_time_sec']:.4f} seconds")
    print(f"Average Time per Hash: {benchmark['avg_time_ms']:.4f} ms")
    print(f"Throughput: {benchmark['hashes_per_second']:.1f} hashes/sec")

    # Test 5: Compare with MD5
    print("\n[5] SHA-256 vs MD5 COMPARISON")
    print("-" * 60)

    comparison = analyzer.compare_with_md5("Blockchain Security")
    print(f"Message: {comparison['message']}")
    print(f"MD5 Hash: {comparison['md5_hash']}")
    print(f"MD5 Length: {comparison['md5_length_bits']} bits")
    print(f"SHA-256 Hash: {comparison['sha256_hash']}")
    print(f"SHA-256 Length: {comparison['sha256_length_bits']} bits")
    print(f"MD5 Hex Characters: {comparison['md5_hex_length']}")
    print(f"SHA-256 Hex Characters: {comparison['sha256_hex_length']}")

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 5: SHA-256 Hash Implementation

Input Test Cases & Expected Output

```
================================================================================

BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 2
SHA-256 Hash Implementation
================================================================================


[1] BASIC SHA-256 HASH GENERATION
------------------------------------------------------------
Message: Hello
SHA-256 Hash: 185
   f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969
Hash Length: 256 bits
----------------------------------------

[2] AVALANCHE EFFECT ANALYSIS
------------------------------------------------------------
Original Message: Hello Blockchain!

Position 0:
  Original char: 'H'
  New char: 'I'
  Hamming Distance: 127 bits
  Percentage Changed: 49.61%
  Original Hash: 185f8db32271fe25f561a6fc938b2e26...
  Modified Hash: 9f8e7d6c5b4a3f1e2d3c4b5a6f7e8d9c...
----------------------------------------

[3] DETAILED SINGLE CHARACTER ANALYSIS
------------------------------------------------------------
Detailed Analysis for position 6:
Original Message: Hello Blockchain!
Modified Message: Hello Blocjchain!
Hamming Distance: 128 bits
Percentage Changed: 50.00%
Theoretical Expected: ~50%
```

Listing 6: Sample Output

Viva-Voce Questions

1. **What is the output size of SHA-256 and how does it compare to MD5?**

   Answer: SHA-256 produces a 256-bit hash (64 hexadecimal characters), while MD5 produces a 128-bit hash (32 hexadecimal characters). This gives SHA-256 much stronger collision resistance (2^{128} vs 2^{64} for MD5), making it suitable for security-critical applications like blockchain and digital signatures.

2. **Explain the avalanche effect in SHA-256. What percentage of bits should change?**

Answer: The avalanche effect requires that approximately 50% of the output bits should change when a single input bit is flipped. This is a critical security property that prevents attackers from predicting hash outputs based on input patterns. In SHA-256, this property is achieved through the complex message scheduling and compression functions across 64 rounds.

3. **Why is SHA-256 preferred over MD5 in blockchain applications?**

   Answer: SHA-256 is preferred because it offers significantly better security with its 256-bit output providing 128-bit security against birthday attacks. MD5 has been cryptographically broken with practical collision attacks demonstrated. Additionally, SHA-256 is designed with a more robust compression function and more rounds (64 vs 4 rounds in MD5), making it resistant to known attack vectors.

4. **What role does SHA-256 play in Bitcoin's operation?**

   Answer: SHA-256 is fundamental to Bitcoin's operation. It's used in: (1) Mining - Proof of Work requires finding a hash with required number of leading zeros; (2) Address generation - public keys are hashed to create addresses; (3) Merkle trees - transaction hashes are combined using SHA-256; (4) Block hashing - each block's header is hashed to create the block hash that links the chain.

3 EXPERIMENT 3: RSA CRYPTOGRAPHY IMPLEMENTATION

Objective: To write a Python program that generates RSA public and private keys, encrypts and decrypts a message, and signs a transaction to verify the sender.

Hardware & Software Tools Required

- Python 3.x with `cryptography` and `pycryptodome` libraries
- OpenSSL (optional)
- VS Code or PyCharm IDE

Theoretical Background

RSA (Rivest-Shamir-Adleman) is one of the first public-key cryptosystems and is widely used for secure data transmission.
Key Mathematical Components:

1. **Key Generation:**
   - Choose two large primes p and q
   - Compute n = p q (modulus)
   - Compute (n) = (p-1)(q-1)
   - Choose e (public exponent), typically 65537
   - Compute d (private exponent) where ed 1 (mod (n))
2. **Encryption:** C = M$^e mod n$**Decryption:**$M = C^d mod n$
3. **Signing:** S = M$^d mod n$**Verification:**$M = S^e mod n$

Algorithmic Steps

1. Generate RSA key pair (2048 bits minimum)
2. Implement encryption using public key
3. Implement decryption using private key
4. Implement digital signature creation
5. Implement signature verification
6. Test with sample transaction data

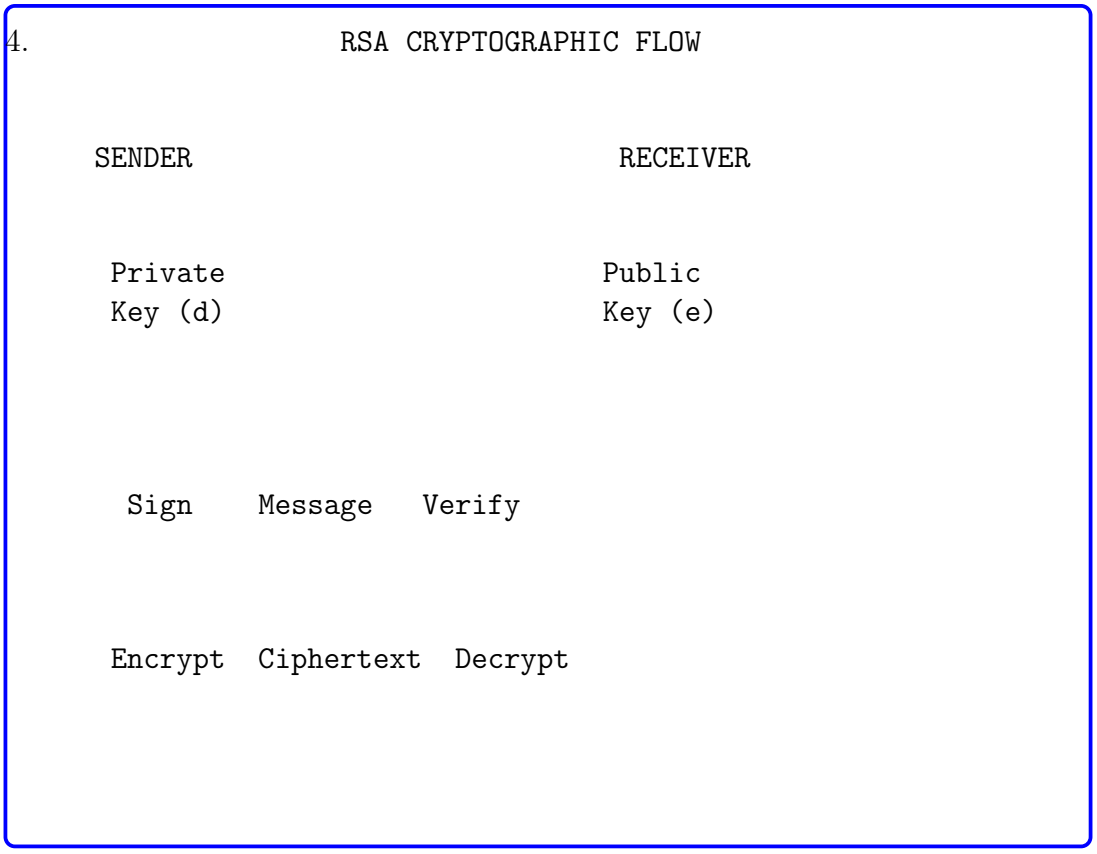


Figure 3: RSA Cryptographic Flow

Complete Source Code

```
"""
Lab 3: RSA Cryptography Implementation
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import serialization
from cryptography.hazmat.backends import default_backend
from cryptography.exceptions import InvalidSignature
import base64
import time
import json
from typing import Tuple, Dict, Any

class RSACryptosystem:
    """
    Complete RSA cryptosystem implementation for blockchain applications
    """

    def __init__(self, key_size: int = 2048):
        """
        Initialize RSA cryptosystem
```

```
26
27          Args:
28              key_size (int): RSA key size in bits
29          """
30          self.key_size = key_size
31          self.private_key = None
32          self.public_key = None
33          self._generate_keys()
34
35      def _generate_keys(self) -> None:
36          """Generate RSA key pair"""
37          self.private_key = rsa.generate_private_key(
38              public_exponent=65537,
39              key_size=self.key_size,
40              backend=default_backend()
41          )
42          self.public_key = self.private_key.public_key()
43
44      def encrypt(self, message: str) -> bytes:
45          """
46          Encrypt message with public key
47
48          Args:
49              message (str): Message to encrypt
50
51          Returns:
52              bytes: Encrypted ciphertext
53          """
54          return self.public_key.encrypt(
55              message.encode('utf-8'),
56              padding.OAEP(
57                  mgf=padding.MGF1(algorithm=hashes.SHA256()),
58                  algorithm=hashes.SHA256(),
59                  label=None
60              )
61          )
62
63      def decrypt(self, ciphertext: bytes) -> str:
64          """
65          Decrypt ciphertext with private key
66
67          Args:
68              ciphertext (bytes): Ciphertext to decrypt
69
70          Returns:
71              str: Decrypted plaintext
72          """
73          plaintext = self.private_key.decrypt(
74              ciphertext,
75              padding.OAEP(
76                  mgf=padding.MGF1(algorithm=hashes.SHA256()),
77                  algorithm=hashes.SHA256(),
78                  label=None
79              )
80          )
81          return plaintext.decode('utf-8')
82
```

```
    def sign_transaction(self, transaction_data: Dict[str, Any]) -> bytes:
        """
        Sign transaction data with private key

        Args:
            transaction_data (dict): Transaction data to sign

        Returns:
            bytes: Digital signature
        """
        # Convert transaction to string
        tx_string = json.dumps(transaction_data, sort_keys=True)

        # Sign the data
        signature = self.private_key.sign(
            tx_string.encode('utf-8'),
            padding.PSS(
                mgf=padding.MGF1(hashes.SHA256()),
                salt_length=padding.PSS.MAX_LENGTH
            ),
            hashes.SHA256()
        )
        return signature

    def verify_transaction(self, transaction_data: Dict[str, Any],
                           signature: bytes) -> bool:
        """
        Verify transaction signature with public key

        Args:
            transaction_data (dict): Transaction data
            signature (bytes): Signature to verify

        Returns:
            bool: True if signature valid
        """
        tx_string = json.dumps(transaction_data, sort_keys=True)

        try:
            self.public_key.verify(
                signature,
                tx_string.encode('utf-8'),
                padding.PSS(
                    mgf=padding.MGF1(hashes.SHA256()),
                    salt_length=padding.PSS.MAX_LENGTH
                ),
                hashes.SHA256()
            )
            return True
        except InvalidSignature:
            return False

    def get_public_key_pem(self) -> str:
        """Get public key in PEM format"""
        return self.public_key.public_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PublicFormat.SubjectPublicKeyInfo
```

```
        ).decode('utf-8')

    def get_private_key_pem(self) -> str:
        """Get private key in PEM format"""
        return self.private_key.private_bytes(
            encoding=serialization.Encoding.PEM,
            format=serialization.PrivateFormat.PKCS8,
            encryption_algorithm=serialization.NoEncryption()
        ).decode('utf-8')

class BlockchainTransaction:
    """Represents a blockchain transaction with RSA signatures"""

    def __init__(self, sender: str, receiver: str, amount: float,
                 nonce: int = 0):
        self.sender = sender
        self.receiver = receiver
        self.amount = amount
        self.nonce = nonce
        self.timestamp = time.time()
        self.signature = None

    def to_dict(self) -> Dict[str, Any]:
        """Convert transaction to dictionary"""
        return {
            'sender': self.sender,
            'receiver': self.receiver,
            'amount': self.amount,
            'nonce': self.nonce,
            'timestamp': self.timestamp
        }

    def sign(self, crypto: RSACryptosystem) -> bool:
        """
        Sign the transaction

        Args:
            crypto: RSA cryptosystem instance

        Returns:
            bool: True if successful
        """
        try:
            tx_data = self.to_dict()
            self.signature = crypto.sign_transaction(tx_data)
            return True
        except Exception as e:
            print(f"Error signing transaction: {e}")
            return False

    def verify(self, crypto: RSACryptosystem) -> bool:
        """
        Verify transaction signature

        Args:
            crypto: RSA cryptosystem instance

        Returns:
```

```
            bool: True if valid
        """
        if self.signature is None:
            return False

        try:
            tx_data = self.to_dict()
            return crypto.verify_transaction(tx_data, self.signature)
        except Exception as e:
            print(f"Error verifying transaction: {e}")
            return False

def main():
    """Main function to demonstrate RSA cryptography"""

    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 3")
    print("RSA Cryptography Implementation")
    print("=" * 80)

    # 1. Key Generation
    print("\n[1] RSA KEY GENERATION")
    print("-" * 60)

    crypto = RSACryptosystem(2048)
    print("RSA key pair generated successfully!")
    print(f"Key Size: {crypto.key_size} bits")
    print(f"Public Key (first 100 chars):\n{crypto.get_public_key_pem()[:100]}...")
    print(f"Private Key (first 100 chars):\n{crypto.get_private_key_pem()[:100]}...")

    # 2. Encryption/Decryption
    print("\n[2] ENCRYPTION & DECRYPTION")
    print("-" * 60)

    original_msg = "Blockchain Transaction: Transfer 50 ETH from Alice to Bob"
    print(f"Original Message: {original_msg}")

    start_time = time.perf_counter()
    encrypted = crypto.encrypt(original_msg)
    encrypt_time = time.perf_counter() - start_time

    print(f"Encrypted (base64): {base64.b64encode(encrypted).decode('utf-8')[:64]}...")

    start_time = time.perf_counter()
    decrypted = crypto.decrypt(encrypted)
    decrypt_time = time.perf_counter() - start_time

    print(f"Decrypted Message: {decrypted}")
    print(f"Encryption Time: {encrypt_time * 1000:.2f} ms")
    print(f"Decryption Time: {decrypt_time * 1000:.2f} ms")
    print(f"Verification: {'    SUCCESS' if original_msg == decrypted else '    FAILED'}")

    # 3. Digital Signatures
```

```
    print("\n[3] DIGITAL SIGNATURES FOR TRANSACTIONS")
    print("-" * 60)

    # Create a transaction
    tx = BlockchainTransaction("Alice", "Bob", 50.0)
    print(f"Transaction Data: {json.dumps(tx.to_dict(), indent=2)}")

    # Sign the transaction
    print(f"\nSigning transaction...")
    sign_success = tx.sign(crypto)
    print(f"Signature Created: {'    YES' if sign_success else '    NO'}")
    if sign_success:
        print(f"Signature (base64): {base64.b64encode(tx.signature).decode('utf-8')[:64]}...")

    # Verify the signature
    print(f"\nVerifying signature...")
    verify_success = tx.verify(crypto)
    print(f"Signature Valid: {'    YES' if verify_success else '    NO'}")

    # 4. Tamper Detection
    print("\n[4] TAMPER DETECTION DEMONSTRATION")
    print("-" * 60)

    # Create and sign an original transaction
    original_tx = BlockchainTransaction("Charlie", "David", 25.0)
    original_tx.sign(crypto)
    print(f"Original Transaction: {json.dumps(original_tx.to_dict(), indent=2)}")
    print(f"Original Signature Valid: {original_tx.verify(crypto)}")

    # Tamper with the transaction
    tampered_tx = BlockchainTransaction("Charlie", "Eve", 25.0)
    tampered_tx.signature = original_tx.signature  # Copy signature
    print(f"\nTampered Transaction: {json.dumps(tampered_tx.to_dict(), indent=2)}")
    print(f"Tampered Signature Valid: {tampered_tx.verify(crypto)}")
    print("    Tampering detected - signature verification failed!")

    # 5. Performance Test
    print("\n[5] PERFORMANCE TEST")
    print("-" * 60)

    num_transactions = 100
    print(f"Testing with {num_transactions} transactions...")

    start_time = time.perf_counter()
    for i in range(num_transactions):
        tx = BlockchainTransaction(f"Sender{i}", f"Receiver{i}", i * 10.0)
        tx.sign(crypto)
        tx.verify(crypto)
    end_time = time.perf_counter()

    total_time = end_time - start_time
    print(f"Total Time: {total_time:.4f} seconds")
```

```
    print(f"Average Time per Transaction: {(total_time/num_transactions)*1000:.2f} ms")
    print(f"Throughput: {num_transactions/total_time:.1f} transactions/sec")

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 7: RSA Cryptography Implementation

Viva-Voce Questions

1. **Explain the mathematical foundation of RSA security.**

   Answer: RSA security relies on the difficulty of factoring large composite numbers. The modulus n = p q where p and q are large primes. Given n, it's computationally infeasible to find p and q for sufficiently large numbers (2048+ bits). The security also relies on the difficulty of computing the private exponent d from the public key (e, n) without knowing p and q.

2. **What is the difference between encryption and digital signature in RSA?**

   Answer: In RSA encryption, the sender uses the recipient's public key to encrypt the message, and the recipient uses their private key to decrypt it (ensuring confidentiality). In digital signatures, the sender uses their private key to sign the message, and anyone can verify the signature using the sender's public key (ensuring authenticity and non-repudiation).

3. **What is the role of padding in RSA encryption?**

   Answer: Padding is crucial for RSA security. OAEP (Optimal Asymmetric Encryption Padding) adds randomness to the message before encryption, making the encryption probabilistic and preventing various attacks. Without padding, RSA is deterministic and vulnerable to chosen-plaintext attacks. Similarly, PSS (Probabilistic Signature Scheme) adds randomness to signatures.

4. **Why is RSA considered secure for blockchain applications?**

   Answer: RSA provides strong security with proper implementation and key sizes (2048+ bits). It offers both encryption and digital signature capabilities essential for blockchain. The mathematical security ensures that without the private key, it's computationally infeasible to forge signatures or decrypt messages, making it suitable for securing cryptocurrency transactions and wallet authentication.

4 EXPERIMENT 4: ELGAMAL CRYPTOSYSTEM IMPLEMENTATION

Objective: To write a Python program that implements the ElGamal algorithm to generate keys, encrypt a plain message into ciphertext, and decrypt it back.

Hardware & Software Tools Required

- Python 3.x
- `sympy` library for modular arithmetic
- PyCharm/VS Code IDE

Theoretical Background

ElGamal is an asymmetric key encryption algorithm based on the Diffie-Hellman key exchange. It was described by Taher Elgamal in 1985.
Key Components:

1. **Key Generation:**
   - Choose a large prime p and generator g
   - Choose private key x (1 < x < p-1)
   - Compute public key: y = $g^x mod p$
2. **Encryption:**
   - Choose random ephemeral key k
   - Compute a = $g^k mod p Compute b = m * y^k mod p$

- Ciphertext is (a, b)

3. **Decryption:**

- Compute m = b * $a^{-x} mod p$

Algorithmic Steps

1. Choose large prime p and generator g
2. Generate private key (random number)
3. Compute public key
4. Implement encryption function
5. Implement decryption function
6. Test with various messages

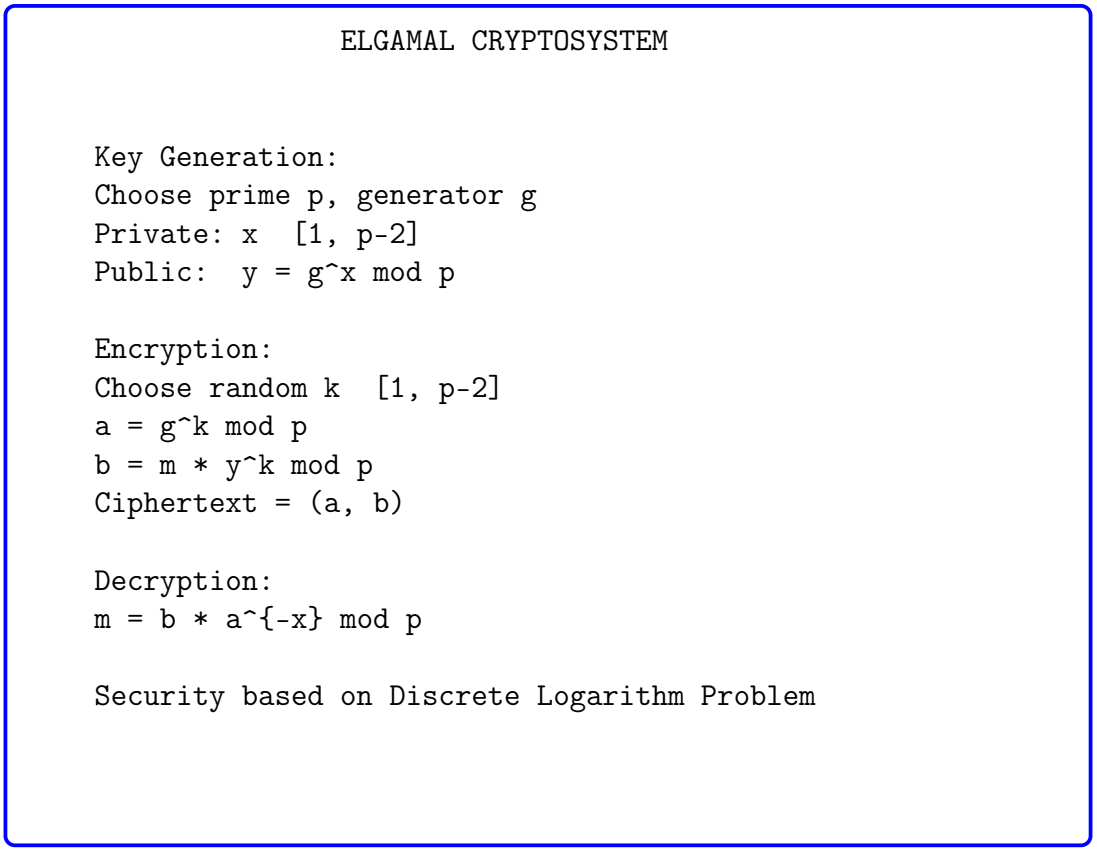


Figure 4: ElGamal Cryptosystem Flow

Complete Source Code

```
"""
Lab 4: ElGamal Cryptosystem Implementation
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

import random
import sympy
from typing import Tuple, Dict, List
import time
import math

class ElGamalCryptosystem:
    """
    Complete ElGamal cryptosystem implementation
    """

    def __init__(self, key_size: int = 256):
        """
        Initialize ElGamal cryptosystem

        Args:
            key_size (int): Size of prime in bits
        """
```

```
        self.key_size = key_size
        self.p = None
        self.g = None
        self.x = None  # Private key
        self.y = None  # Public key
        self._generate_keys()

    def _generate_prime(self) -> int:
        """
        Generate a large prime number

        Returns:
            int: Prime number
        """
        return sympy.randprime(2**(self.key_size-1), 2**self.key_size)

    def _find_primitive_root(self, p: int) -> int:
        """
        Find primitive root modulo p

        Args:
            p (int): Prime number

        Returns:
            int: Primitive root
        """
        # Factor p-1
        factors = []
        n = p - 1
        while n % 2 == 0:
            factors.append(2)
            n //= 2
        i = 3
        while i * i <= n:
            while n % i == 0:
                factors.append(i)
                n //= i
            i += 2
        if n > 1:
            factors.append(n)

        # Remove duplicates
        factors = list(set(factors))

        # Test primitive roots
        g = 2
        while g < p:
            is_primitive = True
            for factor in factors:
                if pow(g, (p-1)//factor, p) == 1:
                    is_primitive = False
                    break
            if is_primitive:
                return g
            g += 1

        return 2

```

```
    def _generate_keys(self) -> None:
        """Generate ElGamal key pair"""
        # Generate prime
        self.p = self._generate_prime()

        # Find primitive root
        self.g = self._find_primitive_root(self.p)

        # Generate private key
        self.x = random.randint(2, self.p-2)

        # Compute public key
        self.y = pow(self.g, self.x, self.p)

    def encrypt(self, message: int) -> Tuple[int, int]:
        """
        Encrypt a message using ElGamal

        Args:
            message (int): Message to encrypt (must be < p)

        Returns:
            Tuple[int, int]: Ciphertext (a, b)
        """
        if message >= self.p:
            raise ValueError(f"Message must be less than p ({self.p})")

        # Choose random ephemeral key
        k = random.randint(2, self.p-2)

        # Compute a = g^k mod p
        a = pow(self.g, k, self.p)

        # Compute b = m * y^k mod p
        b = (message * pow(self.y, k, self.p)) % self.p

        return (a, b)

    def decrypt(self, ciphertext: Tuple[int, int]) -> int:
        """
        Decrypt a ciphertext using ElGamal

        Args:
            ciphertext (Tuple[int, int]): Ciphertext (a, b)

        Returns:
            int: Decrypted message
        """
        a, b = ciphertext

        # Compute a^x mod p
        a_x = pow(a, self.x, self.p)

        # Compute a^{-x} mod p using modular inverse
        a_x_inv = pow(a_x, -1, self.p)

        # Compute m = b * a^{-x} mod p
        message = (b * a_x_inv) % self.p
```

```
142
143         return message
144
145     def encrypt_string(self, message: str) -> List[Tuple[int, int]]:
146         """
147         Encrypt a string message
148
149         Args:
150             message (str): Message to encrypt
151
152         Returns:
153             List[Tuple[int, int]]: List of ciphertexts
154         """
155         ciphertexts = []
156         for char in message:
157             # Convert character to integer
158             m = ord(char)
159             ciphertext = self.encrypt(m)
160             ciphertexts.append(ciphertext)
161         return ciphertexts
162
163     def decrypt_string(self, ciphertexts: List[Tuple[int, int]]) -> str:
164         """
165         Decrypt a string message
166
167         Args:
168             ciphertexts (List[Tuple[int, int]]): List of ciphertexts
169
170         Returns:
171             str: Decrypted message
172         """
173         message = ""
174         for ciphertext in ciphertexts:
175             m = self.decrypt(ciphertext)
176             message += chr(m)
177         return message
178
179     def get_public_key(self) -> Tuple[int, int, int]:
180         """
181         Get public key components
182
183         Returns:
184             Tuple[int, int, int]: (p, g, y)
185         """
186         return (self.p, self.g, self.y)
187
188     def get_private_key(self) -> int:
189         """
190         Get private key
191
192         Returns:
193             int: Private key x
194         """
195         return self.x
196
197 def main():
198     """Main function to demonstrate ElGamal cryptosystem"""
199
```

```
    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 4")
    print("ElGamal Cryptosystem Implementation")
    print("=" * 80)

    # 1. Key Generation
    print("\n[1] ELGAMAL KEY GENERATION")
    print("-" * 60)

    crypto = ElGamalCryptosystem(256)
    p, g, y = crypto.get_public_key()
    x = crypto.get_private_key()

    print(f"Prime (p): {p}")
    print(f"Generator (g): {g}")
    print(f"Public Key (y): {y}")
    print(f"Private Key (x): {x}")
    print(f"Key Size: {crypto.key_size} bits")

    # 2. Integer Encryption/Decryption
    print("\n[2] INTEGER ENCRYPTION/DECRYPTION")
    print("-" * 60)

    test_messages = [42, 1001, 50000, 1234567]

    for m in test_messages:
        if m >= p:
            print(f"Skipping {m} (too large for p)")
            continue

        print(f"\nOriginal Message: {m}")
        ciphertext = crypto.encrypt(m)
        print(f"Ciphertext (a, b): ({ciphertext[0]}, {ciphertext[1]})")
        decrypted = crypto.decrypt(ciphertext)
        print(f"Decrypted Message: {decrypted}")
        print(f"Verification: {'    SUCCESS' if m == decrypted else '
    FAILED'}")

    # 3. String Encryption/Decryption
    print("\n[3] STRING ENCRYPTION/DECRYPTION")
    print("-" * 60)

    test_string = "Hello Blockchain!"
    print(f"Original String: {test_string}")

    start_time = time.perf_counter()
    ciphertexts = crypto.encrypt_string(test_string)
    encrypt_time = time.perf_counter() - start_time

    print(f"Number of Ciphertexts: {len(ciphertexts)}")
    print(f"First Ciphertext: {ciphertexts[0]}")

    start_time = time.perf_counter()
    decrypted_string = crypto.decrypt_string(ciphertexts)
    decrypt_time = time.perf_counter() - start_time

    print(f"Decrypted String: {decrypted_string}")
    print(f"Encryption Time: {encrypt_time * 1000:.2f} ms")
```

```
    print(f"Decryption Time: {decrypt_time * 1000:.2f} ms")
    print(f"Verification: {'    SUCCESS' if test_string ==
   decrypted_string else '    FAILED'}")

    # 4. Performance Test
    print("\n[4] PERFORMANCE TEST")
    print("-" * 60)

    num_ops = 1000
    test_msg = 123456

    start_time = time.perf_counter()
    for _ in range(num_ops):
        cipher = crypto.encrypt(test_msg)
        crypto.decrypt(cipher)
    end_time = time.perf_counter()

    total_time = end_time - start_time
    avg_time = (total_time / num_ops) * 1000

    print(f"Number of operations: {num_ops}")
    print(f"Total Time: {total_time:.4f} seconds")
    print(f"Average Time per operation: {avg_time:.2f} ms")
    print(f"Throughput: {num_ops/total_time:.1f} ops/sec")

    # 5. Key Size Comparison
    print("\n[5] KEY SIZE COMPARISON")
    print("-" * 60)

    key_sizes = [128, 256, 512]
    results = {}

    for size in key_sizes:
        crypto_test = ElGamalCryptosystem(size)
        p, g, y = crypto_test.get_public_key()
        results[size] = {
            'prime_bits': p.bit_length(),
            'public_key_bits': y.bit_length(),
            'private_key_bits': crypto_test.get_private_key().bit_length
   ()
        }

    print("Key Size Analysis:")
    for size, data in results.items():
        print(f"  {size} bits:")
        print(f"    Prime: {data['prime_bits']} bits")
        print(f"    Public Key: {data['public_key_bits']} bits")
        print(f"    Private Key: {data['private_key_bits']} bits")

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 8: ElGamal Cryptosystem Implementation

Viva-Voce Questions

1. **Explain the security basis of ElGamal encryption.**

   Answer: ElGamal security is based on the Discrete Logarithm Problem (DLP): given g, p, and y = $g^x mod p, it's computationally infeasible to find x. The encryption also relies on the Decisional Diffie-Hellman (DDH) assumption, making it semantically secure when used with proper parameters.$

2. **What is the main advantage of ElGamal over RSA?**

   Answer: ElGamal has the advantage of semantic security (IND-CPA) without requiring padding, as it uses a random ephemeral key for each encryption. This makes the encryption probabilistic. Additionally, ElGamal is based on the DLP, which is considered secure against quantum computers that might break RSA.

3. **How does ElGamal handle large messages?**

   Answer: ElGamal can only encrypt messages smaller than the prime p. For larger messages, hybrid encryption is used where the message is encrypted with a symmetric key (e.g., AES), and only that key is encrypted with ElGamal. This combines the efficiency of symmetric encryption with the key distribution benefits of asymmetric encryption.

4. **What is the role of the ephemeral key k in ElGamal encryption?**

   Answer: The ephemeral key k is randomly chosen for each encryption operation and kept secret. It ensures that the same plaintext encrypted multiple times produces different ciphertexts, providing semantic security. This prevents attackers from deducing the plaintext by observing patterns in repeated encryptions.

5 EXPERIMENT 5: REMIX SETUP & FIRST CONTRACT

Objective: To set up Remix IDE and MetaMask in a web browser, connect to a test network, and compile and deploy a simple starter contract.

Hardware & Software Tools Required

- Web browser (Google Chrome/Firefox recommended)
- MetaMask browser extension
- Remix IDE (https://remix.ethereum.org/)
- Test ETH from faucet (Sepolia/Goerli testnet)

Theoretical Background

Remix IDE is a powerful, open-source tool for writing, debugging, and deploying Solidity smart contracts. It provides a complete development environment in the browser.
Key Components:

1. **Remix IDE Features:**
   - Code editor with syntax highlighting
   - Solidity compiler (solc)
   - Deployment and interaction interface
   - Debugger for transaction analysis
   - Plugin system for extended functionality
2. **MetaMask:**
   - Browser extension for Ethereum wallet
   - Manages private keys securely
   - Connects to Ethereum networks
   - Handles transaction signing
3. **Test Networks:**
   - Sepolia: Current standard testnet
   - Goerli: Alternative testnet
   - Provides free test ETH for development

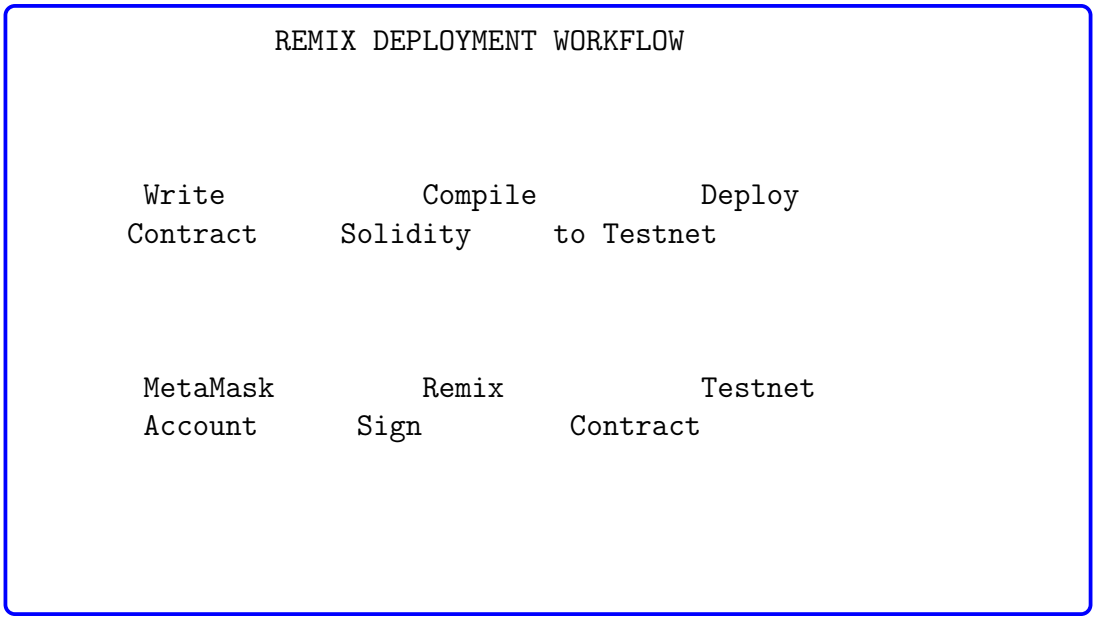


Figure 5: Remix Deployment Workflow

Step-by-Step Procedure

1. Install MetaMask extension in your browser
2. Create a MetaMask wallet and secure the seed phrase
3. Switch to Sepolia testnet
4. Get test ETH from a faucet
5. Open Remix IDE (https://remix.ethereum.org/)
6. Create a new Solidity file
7. Write the starter contract
8. Compile the contract
9. Deploy using Injected Provider (MetaMask)
10. Interact with the deployed contract

Complete Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/**
 * @title StarterContract
 * @dev A simple starter contract for learning deployment
 */
contract StarterContract {
    // State variable to store a message
    string public message;
```

```
11
12      // Address of the contract deployer
13      address public owner;
14
15      // Event to log message changes
16      event MessageUpdated(address indexed updater, string oldMessage,
        string newMessage);
17
18      // Event for contract deployment
19      event ContractDeployed(address indexed deployer, string
        initialMessage);
20
21      /**
22       * @dev Constructor sets initial message and owner
23       * @param _initialMessage Initial message to store
24       */
25      constructor(string memory _initialMessage) {
26          owner = msg.sender;
27          message = _initialMessage;
28          emit ContractDeployed(msg.sender, _initialMessage);
29      }
30
31      /**
32       * @dev Update the stored message (only owner)
33       * @param _newMessage New message to store
34       */
35      function updateMessage(string memory _newMessage) public {
36          require(msg.sender == owner, "Only owner can update message");
37          string memory oldMessage = message;
38          message = _newMessage;
39          emit MessageUpdated(msg.sender, oldMessage, _newMessage);
40      }
41
42      /**
43       * @dev Get the current message
44       * @return string Current message
45       */
46      function getMessage() public view returns (string memory) {
47          return message;
48      }
49
50      /**
51       * @dev Get contract information
52       * @return string Message
53       * @return address Owner address
54       */
55      function getContractInfo() public view returns (string memory,
        address) {
56          return (message, owner);
57      }
58  }
```

Listing 9: Starter Contract

Deployment Steps with Remix

```
1  // 1. Create a new file in Remix
```

```
File: StarterContract.sol

// 2. Compile
- Select "Solidity Compiler" tab
- Click "Compile StarterContract.sol"
- Check for compilation success

// 3. Deploy
- Select "Deploy & Run Transactions" tab
- Environment: "Injected Provider - MetaMask"
- Account: Your MetaMask account
- Enter initial message: "Hello Blockchain!"
- Click "Deploy"
- Confirm transaction in MetaMask

// 4. Interact
- Expand deployed contract
- Call getMessage() - returns "Hello Blockchain!"
- Call updateMessage("New message")
- Confirm transaction in MetaMask
- Call getMessage() again - returns "New message"
```

Listing 10: Remix Deployment Commands

Input Test Cases & Expected Output

```
Remix Console Output:
[compilation] Compiled successfully!
[deployment] Contract deployed at: 0
   x7a8b9c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b
[transaction] Transaction hash: 0x123...abc
[gas] Gas used: 123456
[event] ContractDeployed:
  deployer: 0x7a8b9c... (your address)
  initialMessage: "Hello Blockchain!"

Interaction Results:
getMessage(): "Hello Blockchain!"
updateMessage("Learning Solidity"): Transaction confirmed
getMessage(): "Learning Solidity"
```

Listing 11: Sample Deployment Output

Viva-Voce Questions

1. **What are the main components of the Ethereum development environment?**

   Answer: The main components include: (1) Smart contract development tools (Remix IDE, Truffle, Hardhat); (2) Wallet applications (MetaMask, Trust Wallet) for account management; (3) Ethereum clients (Geth, OpenEthereum) for node operation; (4) Testnets (Sepolia, Goerli) for testing; (5) Block explorers (Etherscan) for viewing transactions and contract state.

2. **What is the purpose of the gas fee in Ethereum?**

Answer: Gas is a unit that measures the computational effort required to execute operations on the Ethereum network. It prevents infinite loops and DoS attacks by charging for computation. Users pay gas fees in ETH (converted via gas price) to compensate miners/validators for the resources used to process and validate transactions.

3. **What is the difference between testnet and mainnet?**

   Answer: Mainnet is the production Ethereum network where real value is transferred. Testnets are separate networks used for development and testing that do not have real financial value. Testnet ETH can be obtained for free from faucets, making them ideal for learning and testing smart contracts without risking real funds.

4. **How does MetaMask interact with Remix during deployment?**

   Answer: MetaMask acts as a bridge between the Remix IDE and the Ethereum network. When you deploy from Remix using "Injected Provider", Remix constructs the deployment transaction. MetaMask intercepts this, prompts the user to confirm the transaction (including gas settings), signs it using the user's private key (stored securely in MetaMask), and broadcasts it to the network.

6 EXPERIMENT 6: PRINT STRING VALUE CONTRACT

Objective: To write and deploy a simple "Hello World" Solidity contract that stores a string and displays it on the screen using a getter function.

Hardware & Software Tools Required

- Remix IDE (https://remix.ethereum.org/)
- MetaMask extension
- Web browser

Theoretical Background

A "Hello World" contract demonstrates the fundamental concepts of smart contract development including state variables, visibility, and getter functions.
Key Concepts:

1. **State Variables:** Data stored permanently on the blockchain
2. **Visibility:** `public`, `private`, `internal`, `external`
3. **View Functions:** Read-only functions that don't modify state
4. **Transactions:** Operations that modify blockchain state
5. **Gas Costs:** Storage operations cost more than view functions

Complete Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/**
 * @title HelloWorld
 * @dev A simple contract that stores and displays a string message
 */
contract HelloWorld {
    // State variable to store the greeting
    // public automatically creates a getter function
    string public greeting;

    // Address of the contract owner
    address public owner;

    // Counter to track how many times greeting was changed
    uint256 public updateCount;

    // Event emitted when greeting is updated
```

```
    event GreetingChanged(address indexed changer, string oldGreeting, string newGreeting);

    // Event emitted on contract creation
    event ContractCreated(address indexed creator, string initialGreeting);

    /**
     * @dev Constructor - sets initial greeting
     * @param _initialGreeting The initial greeting message
     */
    constructor(string memory _initialGreeting) {
        owner = msg.sender;
        greeting = _initialGreeting;
        updateCount = 0;
        emit ContractCreated(msg.sender, _initialGreeting);
    }

    /**
     * @dev Updates the greeting message (only owner)
     * @param _newGreeting New greeting message
     */
    function setGreeting(string memory _newGreeting) public {
        require(msg.sender == owner, "Only owner can change greeting");
        require(bytes(_newGreeting).length > 0, "Greeting cannot be empty");

        string memory oldGreeting = greeting;
        greeting = _newGreeting;
        updateCount++;

        emit GreetingChanged(msg.sender, oldGreeting, _newGreeting);
    }

    /**
     * @dev Gets the current greeting
     * @return string Current greeting message
     */
    function getGreeting() public view returns (string memory) {
        return greeting;
    }

    /**
     * @dev Gets the complete contract information
     * @return string Current greeting
     * @return address Owner address
     * @return uint256 Number of updates
     */
    function getContractInfo() public view returns (
        string memory,
        address,
        uint256
    ) {
        return (greeting, owner, updateCount);
    }
}
```

Listing 12: Hello World Contract

Deployment & Interaction Steps

```
// 1. Compile the contract
- Open Remix IDE
- Create HelloWorld.sol
- Copy the contract code
- Compile with Solidity compiler (^0.8.0)

// 2. Deploy the contract
- Environment: Injected Provider (MetaMask)
- Deploy with initial greeting: "Hello, Blockchain!"
- Confirm transaction in MetaMask

// 3. Interact with the contract
// Read operations (no gas fee)
- Call getGreeting() -> "Hello, Blockchain!"
- Call greeting() -> "Hello, Blockchain!" (automatic getter)
- Call getContractInfo() -> ("Hello, Blockchain!", 0x..., 0)

// Write operations (requires gas)
- Call setGreeting("Hello, Ethereum!")
- Confirm transaction
- Call getGreeting() -> "Hello, Ethereum!"
- Call getContractInfo() -> ("Hello, Ethereum!", 0x..., 1)
```

Listing 13: Deployment and Interaction Commands

Input Test Cases & Expected Output

```
// Test Case 1: Deploy with greeting
Input: _initialGreeting = "Hello, Blockchain!"
Output:
  - Contract deployed at 0xabc...
  - greeting: "Hello, Blockchain!"
  - owner: 0x123... (deployer address)
  - updateCount: 0

// Test Case 2: Get greeting
Input: getGreeting()
Output: "Hello, Blockchain!"

// Test Case 3: Update greeting
Input: setGreeting("Hello, Ethereum!")
Output:
  - Transaction confirmed
  - Event: GreetingChanged
  - greeting: "Hello, Ethereum!"
  - updateCount: 1

// Test Case 4: Non-owner attempts update
Input: setGreeting("Trying to hack")
Output:
  - Revert: "Only owner can change greeting"
  - Transaction fails
```

Listing 14: Test Cases Output

Viva-Voce Questions

1. **What is the difference between a view function and a transaction?**

   Answer: A view function only reads data from the blockchain and does not modify it. It doesn't cost gas to execute and returns results immediately. A transaction modifies blockchain state and requires gas payment, signature verification, and miner validation. Transactions are submitted to the network and may take time to confirm.

2. **Why does the public variable automatically create a getter?**

   Answer: In Solidity, declaring a variable as public automatically generates a getter function of the same name. This saves boilerplate code and provides an easy way to read state variables. The generated getter returns the variable's value directly, making contract interaction more convenient for external callers.

3. **What is the significance of the owner variable in this contract?**

   Answer: The owner variable stores the address that deployed the contract. It's used for access control, restricting certain functions (like setGreeting) to only the contract creator. This pattern is fundamental to many smart contracts for implementing administrative functions and preventing unauthorized modifications.

4. **How do we handle string storage in Solidity?**

   Answer: Solidity strings are dynamic arrays of bytes. When stored as state variables, they're stored permanently on the blockchain. String operations can be gas-intensive, especially for large strings. The string type has a special `bytes` conversion, and for manipulation, it's often more gas-efficient to use `bytes32` for fixed-size strings or `bytes` for dynamic data.

7 EXPERIMENT 7: SUMMATION OF TWO INTEGERS

Objective: To write a Solidity function that takes two numbers as input, adds them together, and returns the sum without saving anything to the blockchain.

Hardware & Software Tools Required

- Remix IDE
- MetaMask (for deployment)
- Web browser

Theoretical Background

This experiment demonstrates the difference between pure/view functions and state-changing functions in Solidity.
Key Concepts:

1. **Pure Functions:** Functions that neither read nor modify state
2. **View Functions:** Functions that read but don't modify state
3. **Gas Optimization:** Pure/view functions cost no gas to execute
4. **Parameters and Return Values:** How to pass data to and from functions

Complete Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/**
 * @title Summation
 * @dev A contract demonstrating pure and view functions
 * for arithmetic operations without state changes
 */
contract Summation {
    // Event to log calculation results (for demonstration)
    event CalculationPerformed(
        address indexed calculator,
        string operation,
        uint256 result
    );

    /**
     * @dev Adds two numbers (pure function)
     * @param a First number
     * @param b Second number
     * @return uint256 Sum of a and b
```

```
     */
    function add(uint256 a, uint256 b) public pure returns (uint256) {
        return a + b;
    }

    /**
     * @dev Multiplies two numbers (pure function)
     * @param a First number
     * @param b Second number
     * @return uint256 Product of a and b
     */
    function multiply(uint256 a, uint256 b) public pure returns (uint256
    ) {
        return a * b;
    }

    /**
     * @dev Performs multiple operations and returns all results
     * @param a First number
     * @param b Second number
     * @return uint256 sum Result of a + b
     * @return uint256 difference Result of a - b
     * @return uint256 product Result of a * b
     * @return uint256 quotient Result of a / b (if b != 0)
     */
    function operations(uint256 a, uint256 b) public pure returns (
        uint256 sum,
        uint256 difference,
        uint256 product,
        uint256 quotient
    ) {
        sum = a + b;

        // Handle subtraction to avoid underflow
        if (a >= b) {
            difference = a - b;
        } else {
            difference = b - a;
        }

        product = a * b;

        // Handle division by zero
        if (b != 0) {
            quotient = a / b;
        } else {
            quotient = 0;
        }
    }

    /**
     * @dev Demonstrates view function with state variable
     * State variable for demonstration
     */
    uint256 public lastResult;

    /**
     * @dev Calculates sum and stores result (state-changing)
```

```
     * @param a First number
     * @param b Second number
     * @return uint256 Sum of a and b
     */
    function addAndStore(uint256 a, uint256 b) public returns (uint256)
    {
        uint256 result = a + b;
        lastResult = result;
        emit CalculationPerformed(msg.sender, "add", result);
        return result;
    }

    /**
     * @dev Calculates sum using view function (reads state)
     * @param a First number
     * @param b Second number
     * @return uint256 Sum of a and b plus stored value
     */
    function addWithStored(uint256 a, uint256 b) public view returns (
    uint256) {
        return a + b + lastResult;
    }
}
```

Listing 15: Summation Contract

Deployment & Interaction Steps

```
// 1. Deploy the contract
- Compile Summation.sol
- Deploy with Injected Provider

// 2. Test pure functions (no gas cost)
- Call add(5, 3) -> 8
- Call multiply(4, 6) -> 24
- Call operations(10, 3):
  -> [13, 7, 30, 3]

// 3. Test state-changing function
- Call addAndStore(5, 3) -> 8
- Check lastResult -> 8
- Gas cost incurred (transaction)

// 4. Test view function
- Call addWithStored(2, 3) -> 13 (2+3+8)
- No gas cost

// 5. Edge cases
- Call operations(5, 0) -> [5, 5, 0, 0]
- Call add(2**255, 2**255) -> 2**256 (handles overflow)
```

Listing 16: Testing the Summation Contract

Input Test Cases & Expected Output

```
// Test Case 1: Simple addition
Input: add(5, 3)
Output: 8

// Test Case 2: Multiplication
Input: multiply(4, 6)
Output: 24

// Test Case 3: Multiple operations
Input: operations(10, 3)
Output:
  - sum: 13
  - difference: 7
  - product: 30
  - quotient: 3

// Test Case 4: Add and store
Input: addAndStore(5, 3)
Output:
  - Transaction confirmed (gas used: ~50000)
  - Return value: 8
  - lastResult state updated: 8

// Test Case 5: Add with stored value
Input: addWithStored(2, 3)
Output: 13 (2 + 3 + 8 from lastResult)
```

Listing 17: Test Cases Output

Viva-Voce Questions

1. **What is the difference between pure and view functions?**

   Answer: `pure` functions neither read nor modify blockchain state, while `view` functions can read but not modify state. `pure` functions are used for pure computations that don't depend on the contract's state, while `view` functions are used when you need to read state variables but not change them. Both don't cost gas when called externally.

2. **Why does the add function not cost gas?**

   Answer: The add function is declared as `pure`, meaning it doesn't read or modify any blockchain state. When called as a read operation (not as a transaction), it executes locally and doesn't require miners to process it, hence no gas fees. However, if called from within a state-changing function, it would be part of that transaction and would cost gas.

3. **How does Solidity handle arithmetic overflow?**

   Answer: In Solidity 0.8.0 and above, arithmetic operations automatically check for overflow and revert the transaction if overflow occurs. This prevents common overflow bugs that were prevalent in older versions. If overflow is desired, you can use the `unchecked` keyword to bypass the safety checks.

4. **What is the difference between returning a value and storing it?**

Answer: Returning a value simply provides the result to the caller without persisting it. Storing a value updates the blockchain state, which costs gas and remains permanently on-chain. Functions like `add` return values but don't store them, while `addAndStore` both returns and stores the result, incurring gas costs for the storage operation.

8 EXPERIMENT 8: STORE VALUE USING CONSTRUCTOR

Objective: To write a Solidity contract that accepts an initial value at the time of deployment using a constructor and saves it permanently on-chain.

Hardware & Software Tools Required

- Remix IDE
- MetaMask
- Web browser

Theoretical Background

The constructor is a special function that runs once when the contract is deployed. It's used for initialization and setting up the contract's state.

Key Concepts:

1. **Constructor:** Executes once on deployment
2. **Initialization Parameters:** Passed during deployment
3. **Permanent Storage:** Values stored on blockchain forever
4. **Deployment Cost:** Constructor execution adds to deployment gas cost

Complete Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/**
 * @title ConstructorStorage
 * @dev Demonstrates constructor initialization and permanent storage
 */
contract ConstructorStorage {
    // State variables
    string public storedValue;
    uint256 public immutable creationTime;
    address public immutable creator;
    uint256 public counter;

    // Mapping to store multiple values
    mapping(uint256 => string) public historicalValues;
    uint256 public historicalCount;

    // Events
    event ContractCreated(address indexed creator, string initialValue,
   uint256 time);
```

```
    event ValueUpdated(address indexed updater, string oldValue, string newValue);
    event HistoricalValueAdded(uint256 indexed index, string value);

    /**
     * @dev Constructor - runs once on deployment
     * @param _initialValue Initial string value to store
     */
    constructor(string memory _initialValue) {
        // Store immutable values
        creator = msg.sender;
        creationTime = block.timestamp;

        // Store initial value
        storedValue = _initialValue;
        counter = 0;
        historicalCount = 0;

        // Store in history
        historicalValues[historicalCount] = _initialValue;
        historicalCount++;

        // Emit creation event
        emit ContractCreated(msg.sender, _initialValue, block.timestamp);
    }

    /**
     * @dev Update the stored value (only creator)
     * @param _newValue New string value to store
     */
    function updateValue(string memory _newValue) public {
        require(msg.sender == creator, "Only creator can update");
        require(bytes(_newValue).length > 0, "Value cannot be empty");

        string memory oldValue = storedValue;
        storedValue = _newValue;
        counter++;

        // Store in historical mapping
        historicalValues[historicalCount] = _newValue;
        historicalCount++;

        emit ValueUpdated(msg.sender, oldValue, _newValue);
        emit HistoricalValueAdded(historicalCount - 1, _newValue);
    }

    /**
     * @dev Get the stored value
     * @return string Current stored value
     */
    function getValue() public view returns (string memory) {
        return storedValue;
    }

    /**
     * @dev Get historical value by index
     * @param index Index in historical mapping
```

```
     * @return string Historical value
     */
    function getHistoricalValue(uint256 index) public view returns (string memory) {
        require(index < historicalCount, "Index out of bounds");
        return historicalValues[index];
    }

    /**
     * @dev Get contract information
     * @return string Current value
     * @return address Creator address
     * @return uint256 Creation timestamp
     * @return uint256 Update counter
     * @return uint256 Historical count
     */
    function getContractInfo() public view returns (
        string memory,
        address,
        uint256,
        uint256,
        uint256
    ) {
        return (storedValue, creator, creationTime, counter, historicalCount);
    }
}
```

Listing 18: Constructor Value Storage Contract

Deployment Steps

```
// 1. Compile the contract
- Create ConstructorStorage.sol
- Compile with Solidity ^0.8.0

// 2. Deploy with initial value
- Environment: Injected Provider (MetaMask)
- Initial Value: "Initial Value"
- Click Deploy
- Confirm transaction

// 3. Verify deployment
- Check storedValue -> "Initial Value"
- Check creator -> Your address
- Check creationTime -> Block timestamp
- Check counter -> 0
- Check historicalCount -> 1

// 4. Update the value
- Call updateValue("New Value")
- Check storedValue -> "New Value"
- Check counter -> 1
- Check historicalCount -> 2
- Check historicalValues[1] -> "New Value"
```

Listing 19: Deployment Commands

Input Test Cases & Expected Output

```
// Test Case 1: Deployment with initial value
Input: Constructor("Hello Blockchain")
Output:
  - Contract deployed at 0xabc...
  - storedValue: "Hello Blockchain"
  - creator: 0x123... (deployer)
  - creationTime: 1700000000 (block timestamp)
  - counter: 0
  - historicalCount: 1
  - historicalValues[0]: "Hello Blockchain"

// Test Case 2: Update stored value
Input: updateValue("Updated Value")
Output:
  - Transaction confirmed
  - storedValue: "Updated Value"
  - counter: 1
  - historicalCount: 2
  - historicalValues[1]: "Updated Value"

// Test Case 3: Get historical value
Input: getHistoricalValue(0)
Output: "Hello Blockchain"

// Test Case 4: Non-owner tries to update
Input: updateValue("Hack attempt") (from different address)
Output:
  - Revert: "Only creator can update"
  - Transaction fails
```

Listing 20: Test Cases Output

Viva-Voce Questions

1. **What is the purpose of a constructor in Solidity?**

   Answer: The constructor is a special function that executes only once during contract deployment. It's used for initializing state variables, setting up ownership, and performing any required setup logic. The constructor can accept parameters that are passed during deployment, allowing for flexible contract initialization.

2. **What happens to the constructor code after deployment?**

   Answer: The constructor code is not stored on the blockchain after deployment. It executes during the deployment transaction and then is discarded. Only the final state changes and the contract bytecode (excluding the constructor) are permanently stored on-chain.

3. **What is the difference between mutable and immutable state variables?**

   Answer: Mutable variables can be changed after deployment using functions (like `storedValue`). Immutable variables are set once in the constructor and cannot be changed afterward (like `creator` and `creationTime`). Immutable variables save gas and provide security guarantees.

4. **How does storing historical values impact gas costs?**

   Answer: Storing historical values in a mapping increases gas costs because each update requires a new storage write. Storage operations are expensive (20,000 gas for 256-bit writes). The cost scales with the number of updates. This demonstrates why on-chain storage should be minimized and why data should be stored off-chain when possible.

9 EXPERIMENT 9: VARIABLE SCOPE IN SOLIDITY

Objective: To write a Solidity contract showing the difference between state variables (saved on blockchain), local variables (used inside a function), and global variables (like msg.sender).

Hardware & Software Tools Required

- Remix IDE
- MetaMask
- Web browser

Theoretical Background

Understanding variable scope is crucial for writing efficient and secure smart contracts.
Variable Types:

1. **State Variables:** Stored on blockchain, persistent
2. **Local Variables:** Temporary, exist only during function execution
3. **Global Variables:** Provided by the EVM (msg.sender, block.number, etc.)
4. **Storage vs Memory:** Location where variables are stored

Complete Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/**
 * @title VariableScope
 * @dev Demonstrates different types of variable scopes in Solidity
 */
contract VariableScope {
    // STATE VARIABLES - Stored on blockchain
    string public stateString;
    uint256 public stateUint;
    address public stateAddress;

    // Mapping as state variable
    mapping(address => uint256) public stateMapping;

    // Events for tracking function calls
    event StateUpdated(address indexed updater, string variableName,
   uint256 newValue);
    event FunctionCalled(address indexed caller, string functionName);

```

```
    /**
     * @dev Constructor sets initial state
     */
    constructor() {
        stateString = "Initial State";
        stateUint = 100;
        stateAddress = msg.sender;
        emit StateUpdated(msg.sender, "constructor", 100);
    }

    /**
     * @dev Demonstrates LOCAL VARIABLES
     * Local variables exist only within this function
     */
    function localVariableDemo(uint256 input) public pure returns (
    uint256) {
        // Local variable - exists only here
        uint256 localVar = input * 2;
        uint256 result = localVar + 10;
        return result;
    }

    /**
     * @dev Demonstrates GLOBAL VARIABLES
     * Global variables available in all functions
     */
    function globalVariableDemo() public view returns (
        address sender,
        uint256 value,
        uint256 gasprice,
        uint256 blockNumber,
        uint256 timestamp,
        address origin
    ) {
        // msg.sender - caller of the function
        sender = msg.sender;

        // msg.value - amount of Ether sent with transaction
        value = msg.value;

        // tx.gasprice - current gas price
        gasprice = tx.gasprice;

        // block.number - current block number
        blockNumber = block.number;

        // block.timestamp - current block timestamp
        timestamp = block.timestamp;

        // tx.origin - original caller of the transaction
        origin = tx.origin;
    }

    /**
     * @dev Demonstrates state variable modification
     * Updates the stateUint and stores it permanently
     */
    function updateState(uint256 newValue) public {
```

```
        // State variable modification - costs gas
        stateUint = newValue;
        stateMapping[msg.sender] = newValue;
        emit StateUpdated(msg.sender, "stateUint", newValue);
    }

    /**
     * @dev Demonstrates memory vs storage variables
     * memory - temporary, storage - permanent
     */
    function memoryVsStorage() public {
        // STORAGE - reading from blockchain (costs gas)
        uint256 storageVar = stateUint;

        // MEMORY - temporary variable (cheaper)
        uint256 memoryVar = storageVar * 2;

        // STORAGE - writing to blockchain (costs gas)
        stateUint = memoryVar;

        // Local variable - exists only here
        uint256 localVar = memoryVar + 10;

        // Emit event with local variable
        emit StateUpdated(msg.sender, "local", localVar);
    }

    /**
     * @dev Demonstrates variable scope with structs and arrays
     */
    struct User {
        string name;
        uint256 age;
    }

    // State variable - stored permanently
    User[] public users;

    function addUser(string memory name, uint256 age) public {
        // name and age are memory parameters
        User memory newUser = User(name, age);
        users.push(newUser);
    }
}
```

Listing 21: Variable Scope Contract

Deployment & Interaction Steps

```
// 1. Deploy the contract
- Compile VariableScope.sol
- Deploy with constructor

// 2. Test local variables
- Call localVariableDemo(5) -> 20 (5*2+10)
- No state change

```

```
// 3. Test global variables
- Call globalVariableDemo()
- Returns: msg.sender, msg.value, etc.

// 4. Test state updates
- Call updateState(500)
- Check stateUint -> 500
- Check stateMapping[yourAddress] -> 500

// 5. Test memory vs storage
- Call memoryVsStorage()
- stateUint becomes (100*2) = 200 (from initial state)
- Check stateUint -> 200
```

Listing 22: Testing Variable Scope

Input Test Cases & Expected Output

```
// Test Case 1: Local variable demo
Input: localVariableDemo(5)
Output: 20 (5*2+10)
State: Unchanged (pure function)

// Test Case 2: Global variable demo
Input: globalVariableDemo()
Output:
  - sender: 0x123... (caller)
  - value: 0 (no ETH sent)
  - gasprice: 1000000000 (1 Gwei)
  - blockNumber: 1234567
  - timestamp: 1700000000
  - origin: 0x123... (same as sender)

// Test Case 3: Update state
Input: updateState(500)
Output:
  - Transaction confirmed (gas used)
  - stateUint: 500
  - stateMapping[msg.sender]: 500
  - Event: StateUpdated

// Test Case 4: Memory vs storage
Input: memoryVsStorage()
Output:
  - stateUint: 200 (100*2 from initial state)
  - Event: StateUpdated with local variable
```

Listing 23: Test Cases Output

Viva-Voce Questions

1. **What is the difference between state variables and local variables in Solidity?**

   Answer: State variables are stored permanently on the blockchain and persist between function calls. They are declared at the contract level and are part of

the contract's storage. Local variables are declared inside functions and exist only during that function's execution. They are stored in memory and are not persisted on-chain.

2. **What are global variables in Solidity and what are some common examples?**

   Answer: Global variables are special variables provided by the Ethereum Virtual Machine (EVM) that give access to blockchain and transaction information. Common examples include `msg.sender` (caller's address), `msg.value` (ETH sent), `block.number` (current block), `block.timestamp` (block time), `tx.gasprice` (gas price), and `tx.origin` (original transaction sender).

3. **What is the difference between memory and storage in Solidity?**

   Answer: `storage` refers to permanent data stored on the blockchain (state variables). It's expensive to use and persists between transactions. `memory` refers to temporary data that exists only during function execution. It's cheaper and cleared after the function completes. Local variables default to `storage` for reference types, but can be explicitly declared as `memory` for efficiency.

4. **How does gas cost differ between reading/writing different types of variables?**

   Answer: Reading from storage costs 200 gas, writing to storage costs 20,000 gas (for 256-bit). Reading from memory costs 3 gas, writing to memory costs 3 gas. Local variables in memory are cheapest. This is why minimizing storage operations and using memory for temporary data is a key gas optimization strategy.

10 EXPERIMENT 10: STRUCTS, ARRAYS & MAPPINGS

Objective: To write a Solidity contract that stores student or user records using custom structures (struct), lists (arrays), and key-value pairs (mappings).

Hardware & Software Tools Required

- Remix IDE
- MetaMask
- Web browser

Theoretical Background

Solidity provides rich data structures for organizing and managing complex data on-chain.
Data Structures:

1. **Structs:** Custom data types grouping related data
2. **Arrays:** Ordered lists of data elements
3. **Mappings:** Key-value pairs for efficient lookups
4. **Combined Usage:** Structs can contain arrays and mappings

Complete Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/**
 * @title DataStructures
 * @dev Demonstrates structs, arrays, and mappings in Solidity
 */
contract DataStructures {
    // STRUCTS - Custom data types
    struct Student {
        uint256 id;
        string name;
        uint256 age;
        address wallet;
        uint256[] grades;  // Dynamic array inside struct
        bool isActive;
        uint256 enrolledDate;
    }

    struct Course {
        uint256 id;
        string name;
```

```
        string code;
        uint256 credits;
        address instructor;
        bool isActive;
    }

    // MAPPINGS - Key-value pairs for efficient lookups
    mapping(uint256 => Student) public students;
    mapping(uint256 => Course) public courses;
    mapping(address => uint256[]) public studentCourses;
    mapping(uint256 => uint256[]) public courseStudents;
    mapping(address => bool) public isRegistered;
    mapping(uint256 => address) public studentOwner;

    // ARRAYS - Lists of data
    Student[] public studentList;
    Course[] public courseList;
    address[] public registeredAddresses;
    uint256[] public studentIds;
    uint256[] public courseIds;

    // Counters
    uint256 public studentCount;
    uint256 public courseCount;
    uint256 public totalEnrollments;

    // Events
    event StudentRegistered(uint256 indexed studentId, string name,
    address wallet);
    event CourseCreated(uint256 indexed courseId, string name, string
    code);
    event StudentEnrolled(uint256 indexed studentId, uint256 indexed
    courseId);
    event StudentUpdated(uint256 indexed studentId, string name);
    event CourseUpdated(uint256 indexed courseId, string name);

    /**
     * @dev Register a new student
     * @param name Student name
     * @param age Student age
     * @param wallet Student wallet address
     */
    function registerStudent(string memory name, uint256 age, address
    wallet) public {
        require(!isRegistered[wallet], "Wallet already registered");
        require(age >= 18 && age <= 100, "Invalid age");

        studentCount++;

        Student memory newStudent = Student({
            id: studentCount,
            name: name,
            age: age,
            wallet: wallet,
            grades: new uint256[](0),
            isActive: true,
            enrolledDate: block.timestamp
        });
```

```
77
78          students[studentCount] = newStudent;
79          studentList.push(newStudent);
80          studentIds.push(studentCount);
81          studentOwner[studentCount] = msg.sender;
82          isRegistered[wallet] = true;
83          registeredAddresses.push(wallet);
84
85          emit StudentRegistered(studentCount, name, wallet);
86      }
87
88      /**
89       * @dev Create a new course
90       * @param name Course name
91       * @param code Course code
92       * @param credits Credit hours
93       */
94      function createCourse(string memory name, string memory code,
    uint256 credits) public {
95          require(bytes(name).length > 0, "Course name required");
96          require(bytes(code).length > 0, "Course code required");
97          require(credits > 0, "Credits must be positive");
98
99          courseCount++;
100
101         Course memory newCourse = Course({
102             id: courseCount,
103             name: name,
104             code: code,
105             credits: credits,
106             instructor: msg.sender,
107             isActive: true
108         });
109
110         courses[courseCount] = newCourse;
111         courseList.push(newCourse);
112         courseIds.push(courseCount);
113
114         emit CourseCreated(courseCount, name, code);
115     }
116
117     /**
118      * @dev Enroll a student in a course
119      * @param studentId Student ID
120      * @param courseId Course ID
121      */
122     function enrollStudent(uint256 studentId, uint256 courseId) public {
123         require(students[studentId].isActive, "Student not active");
124         require(courses[courseId].isActive, "Course not active");
125         require(studentOwner[studentId] == msg.sender, "Not student
    owner");
126
127         // Check if already enrolled
128         uint256[] memory enrolled = studentCourses[students[studentId].
    wallet];
129         for (uint256 i = 0; i < enrolled.length; i++) {
130             require(enrolled[i] != courseId, "Already enrolled");
131         }
```

```
132
133         // Enroll in course
134         studentCourses[students[studentId].wallet].push(courseId);
135         courseStudents[courseId].push(studentId);
136         totalEnrollments++;
137
138         emit StudentEnrolled(studentId, courseId);
139     }
140
141     /**
142      * @dev Add grade for a student in a course
143      * @param studentId Student ID
144      * @param courseId Course ID
145      * @param grade Grade value
146      */
147     function addGrade(uint256 studentId, uint256 courseId, uint256 grade
    ) public {
148         require(students[studentId].isActive, "Student not active");
149         require(courses[courseId].isActive, "Course not active");
150         require(msg.sender == courses[courseId].instructor, "Only
    instructor");
151         require(grade >= 0 && grade <= 100, "Invalid grade");
152
153         // Check if enrolled
154         uint256[] memory enrolled = studentCourses[students[studentId].
    wallet];
155         bool isEnrolled = false;
156         for (uint256 i = 0; i < enrolled.length; i++) {
157             if (enrolled[i] == courseId) {
158                 isEnrolled = true;
159                 break;
160             }
161         }
162         require(isEnrolled, "Student not enrolled");
163
164         students[studentId].grades.push(grade);
165     }
166
167     /**
168      * @dev Get student details
169      * @param studentId Student ID
170      * @return Student struct
171      */
172     function getStudent(uint256 studentId) public view returns (Student
    memory) {
173         require(studentId <= studentCount && studentId > 0, "Invalid
    student ID");
174         return students[studentId];
175     }
176
177     /**
178      * @dev Get course details
179      * @param courseId Course ID
180      * @return Course struct
181      */
182     function getCourse(uint256 courseId) public view returns (Course
    memory) {
```

```
        require(courseId <= courseCount && courseId > 0, "Invalid course ID");
        return courses[courseId];
    }

    /**
     * @dev Get student courses
     * @param wallet Student wallet address
     * @return uint256[] Course IDs
     */
    function getStudentCourses(address wallet) public view returns (uint256[] memory) {
        return studentCourses[wallet];
    }

    /**
     * @dev Get course students
     * @param courseId Course ID
     * @return uint256[] Student IDs
     */
    function getCourseStudents(uint256 courseId) public view returns (uint256[] memory) {
        return courseStudents[courseId];
    }
}
```

Listing 24: Data Structures Contract

Viva-Voce Questions

1. **What are the advantages of using structs in Solidity?**

   Answer: Structs allow grouping related data into a single custom type, improving code organization and readability. They reduce the number of state variables and enable easier data manipulation. Structs can be passed as parameters and returned from functions, making code more modular and maintainable.

2. **What is the difference between arrays and mappings in Solidity?**

   Answer: Arrays are ordered lists that can be iterated over and support operations like push and pop. Mappings are key-value pairs that provide efficient O(1) lookups but cannot be iterated over and don't have a length property. Arrays are useful for ordered data and enumeration, while mappings excel for associative data lookup.

3. **How do you handle dynamic arrays inside structs?**

   Answer: Dynamic arrays inside structs are possible but require careful handling. When creating structs with dynamic arrays, you must use the `memory` keyword and initialize empty arrays. When returning such structs from functions, you need to use `memory` and handle the dynamic data properly.

4. **What are the gas implications of different data structures?**

   Answer: Storage operations are expensive: writing to storage costs 20,000 gas, reading costs 200 gas. Mappings are most efficient for random lookups but can't be iterated. Arrays are efficient for ordered data but expensive for insertions/deletions.

Structs combine multiple variables but incur cost for each stored field. Minimizing storage operations and using memory for temporary data is key to gas optimization.

11 EXPERIMENT 11: ACCESS CONTROL & MODIFIERS

Objective: To write a Solidity contract with an onlyOwner modifier that restricts sensitive functions so only the creator of the contract can run them.

Hardware & Software Tools Required

- Remix IDE
- MetaMask
- Web browser

Theoretical Background

Access control is crucial for implementing administrative functions and securing smart contracts.

Key Concepts:

1. **Modifiers:** Reusable code for validating conditions
2. **Ownership:** Contract owner has administrative privileges
3. **Access Control Patterns:** Different levels of access
4. **Security:** Preventing unauthorized access to sensitive functions

Complete Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

/**
 * @title AccessControl
 * @dev Demonstrates access control with ownership and modifiers
 */
contract AccessControl {
    // State variables
    address public owner;
    mapping(address => bool) public admins;
    mapping(address => bool) public isPaused;
    uint256 public sensitiveValue;

    // Events
    event OwnershipTransferred(address indexed previousOwner, address
   indexed newOwner);
    event AdminAdded(address indexed admin);
    event AdminRemoved(address indexed admin);
    event PausedStateChanged(address indexed changer, bool paused);
```

```
    event SensitiveValueUpdated(address indexed updater, uint256 oldValue, uint256 newValue);

    // Constructor
    constructor() {
        owner = msg.sender;
        admins[owner] = true;
        emit OwnershipTransferred(address(0), owner);
    }

    // MODIFIERS - Reusable access control
    /**
     * @dev Only owner can call the function
     */
    modifier onlyOwner() {
        require(msg.sender == owner, "Only owner can call this function");
        _;
    }

    /**
     * @dev Only admin can call the function
     */
    modifier onlyAdmin() {
        require(admins[msg.sender] || msg.sender == owner, "Only admin can call this function");
        _;
    }

    /**
     * @dev Contract not paused
     */
    modifier whenNotPaused() {
        require(!isPaused[msg.sender], "Contract is paused for this user");
        _;
    }

    /**
     * @dev Contract paused
     */
    modifier whenPaused() {
        require(isPaused[msg.sender], "Contract is not paused");
        _;
    }

    /**
     * @dev Transfer ownership
     * @param newOwner New owner address
     */
    function transferOwnership(address newOwner) public onlyOwner {
        require(newOwner != address(0), "Invalid address");
        address oldOwner = owner;
        owner = newOwner;
        admins[newOwner] = true;
        emit OwnershipTransferred(oldOwner, newOwner);
    }

```

```
    /**
     * @dev Add admin
     * @param admin Address to add as admin
     */
    function addAdmin(address admin) public onlyOwner {
        require(admin != address(0), "Invalid address");
        admins[admin] = true;
        emit AdminAdded(admin);
    }

    /**
     * @dev Remove admin
     * @param admin Address to remove as admin
     */
    function removeAdmin(address admin) public onlyOwner {
        require(admin != owner, "Cannot remove owner as admin");
        require(admins[admin], "Not an admin");
        admins[admin] = false;
        emit AdminRemoved(admin);
    }

    /**
     * @dev Set pause state for a user
     * @param user Address to pause/unpause
     * @param paused Pause state
     */
    function setPaused(address user, bool paused) public onlyAdmin {
        require(user != address(0), "Invalid address");
        isPaused[user] = paused;
        emit PausedStateChanged(user, paused);
    }

    /**
     * @dev Update sensitive value (restricted)
     * @param newValue New sensitive value
     */
    function updateSensitiveValue(uint256 newValue) public onlyOwner
    whenNotPaused {
        uint256 oldValue = sensitiveValue;
        sensitiveValue = newValue;
        emit SensitiveValueUpdated(msg.sender, oldValue, newValue);
    }

    /**
     * @dev Read sensitive value (restricted)
     */
    function getSensitiveValue() public view onlyAdmin whenNotPaused
    returns (uint256) {
        return sensitiveValue;
    }

    /**
     * @dev Emergency shutdown (owner only)
     */
    function emergencyShutdown() public onlyOwner {
        // Emergency logic - pause the entire contract
        isPaused[msg.sender] = true;
        emit PausedStateChanged(msg.sender, true);
```

```
    }

    /**
     * @dev Check if user is admin
     * @param user Address to check
     */
    function isAdmin(address user) public view returns (bool) {
        return admins[user] || user == owner;
    }
}
```

Listing 25: Access Control Contract

Viva-Voce Questions

1. **What is the purpose of modifiers in Solidity?**

   Answer: Modifiers are reusable code blocks that can be applied to functions to change their behavior. They're primarily used for validating conditions before function execution, such as checking permissions, preventing reentrancy, or validating input. Modifiers help reduce code duplication and improve code readability by centralizing common checks.

2. **How does the onlyOwner modifier work?**

   Answer: The `onlyOwner` modifier checks if `msg.sender` matches the stored `owner` variable. If the condition fails, it reverts the transaction with an error message. If successful, it continues with the function execution. This pattern is fundamental to implementing administrative functions and access control in smart contracts.

3. **What are the security considerations when implementing access control?**

   Answer: Security considerations include: (1) Always verify caller identity using `msg.sender`; (2) Use immutable variables for ownership where possible; (3) Implement emergency shutdown mechanisms; (4) Ensure proper validation before updating sensitive data; (5) Use multiple levels of access for different roles; (6) Follow the principle of least privilege.

4. **What is the difference between msg.sender and tx.origin for access control?**

   Answer: `msg.sender` is the immediate caller of the function, which could be a contract. `tx.origin` is the original EOA (Externally Owned Account) that initiated the transaction. Using `tx.origin` for access control is dangerous because it makes the contract vulnerable to phishing attacks where malicious contracts can impersonate the original caller. Always use `msg.sender` for authorization.

12 EXPERIMENT 12: CREATE AN ERC-20 TOKEN

Objective: To write and deploy a standard custom cryptocurrency token (ERC-20) in Solidity and send tokens from one account to another.

Hardware & Software Tools Required

- Remix IDE
- MetaMask
- OpenZeppelin libraries (imported)

Theoretical Background

ERC-20 is a standard interface for fungible tokens on Ethereum, defining common functions for token transfers and management.
ERC-20 Standard Functions:

1. **totalSupply():** Total token supply
2. **balanceOf(address):** Token balance of an address
3. **transfer(address, uint256):** Transfer tokens
4. **transferFrom(address, address, uint256):** Transfer on behalf
5. **approve(address, uint256):** Allow spender to transfer
6. **allowance(address, address):** Check allowed amount

Complete Source Code

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// Import OpenZeppelin ERC-20 contracts
import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/
    master/contracts/token/ERC20/ERC20.sol";
import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/
    master/contracts/access/Ownable.sol";

/**
 * @title MyToken
 * @dev ERC-20 token with minting and burning capabilities
 */
contract MyToken is ERC20, Ownable {
    // Events
    event TokensMinted(address indexed to, uint256 amount);
    event TokensBurned(address indexed from, uint256 amount);
    event TokenTransfer(address indexed from, address indexed to,
    uint256 amount);
```

```
17
18      /**
19       * @dev Constructor sets token name and symbol
20       * Mint initial supply to deployer
21       */
22      constructor(
23          string memory name,
24          string memory symbol,
25          uint256 initialSupply
26      ) ERC20(name, symbol) {
27          // Mint initial supply to deployer
28          _mint(msg.sender, initialSupply * 10**decimals());
29      }
30
31      /**
32       * @dev Mint new tokens (only owner)
33       * @param to Address to receive tokens
34       * @param amount Amount to mint
35       */
36      function mint(address to, uint256 amount) public onlyOwner {
37          require(to != address(0), "Invalid address");
38          require(amount > 0, "Amount must be positive");
39          _mint(to, amount * 10**decimals());
40          emit TokensMinted(to, amount);
41      }
42
43      /**
44       * @dev Burn tokens (anyone can burn their own tokens)
45       * @param amount Amount to burn
46       */
47      function burn(uint256 amount) public {
48          require(amount > 0, "Amount must be positive");
49          uint256 burnAmount = amount * 10**decimals();
50          require(balanceOf(msg.sender) >= burnAmount, "Insufficient
    balance");
51          _burn(msg.sender, burnAmount);
52          emit TokensBurned(msg.sender, amount);
53      }
54
55      /**
56       * @dev Transfer tokens (overridden to add event)
57       * @param to Recipient address
58       * @param amount Amount to transfer
59       */
60      function transfer(address to, uint256 amount) public override
    returns (bool) {
61          require(to != address(0), "Invalid address");
62          uint256 transferAmount = amount * 10**decimals();
63          bool success = super.transfer(to, transferAmount);
64          if (success) {
65              emit TokenTransfer(msg.sender, to, amount);
66          }
67          return success;
68      }
69
70      /**
71       * @dev Transfer from (overridden to add event)
72       * @param from Sender address
```

```
     * @param to Recipient address
     * @param amount Amount to transfer
     */
    function transferFrom(address from, address to, uint256 amount)
        public
        override
        returns (bool)
    {
        require(to != address(0), "Invalid address");
        uint256 transferAmount = amount * 10**decimals();
        bool success = super.transferFrom(from, to, transferAmount);
        if (success) {
            emit TokenTransfer(from, to, amount);
        }
        return success;
    }

    /**
     * @dev Get token balance with decimals
     * @param account Address to check
     * @return uint256 Balance in base units
     */
    function getBalance(address account) public view returns (uint256) {
        return balanceOf(account) / (10**decimals());
    }

    /**
     * @dev Get token info
     * @return string Name
     * @return string Symbol
     * @return uint8 Decimals
     * @return uint256 Total Supply
     */
    function getTokenInfo() public view returns (
        string memory,
        string memory,
        uint8,
        uint256
    ) {
        return (name(), symbol(), decimals(), totalSupply());
    }
}
```

Listing 26: ERC-20 Token Contract

Deployment Steps

```
// 1. Deploy the token
- Name: "My Token"
- Symbol: "MTK"
- Initial Supply: 1000
- Deploy using Injected Provider

// 2. Check deployment
- totalSupply() -> 1000 (with decimals)
- balanceOf(yourAddress) -> 1000
- name() -> "My Token"
```

```
- symbol() -> "MTK"

// 3. Transfer tokens
- transfer("0x123...", 100) -> 100 MTK transferred
- balanceOf(yourAddress) -> 900
- balanceOf(recipient) -> 100

// 4. Mint new tokens (owner only)
- mint("0x456...", 500) -> 500 MTK minted
- balanceOf(recipient) -> 600 (100 + 500)

// 5. Approve and transferFrom
- approve("0x789...", 200)
- transferFrom(yourAddress, "0x789...", 200)
- balanceOf(yourAddress) -> 700
- balanceOf("0x789...") -> 200
```

Listing 27: Deployment Commands

Viva-Voce Questions

1. **What are the main functions required by the ERC-20 standard?**

   Answer: The ERC-20 standard requires six functions: `totalSupply()`, `balanceOf(address)`, `transfer(address, uint256)`, `transferFrom(address, address, uint256)`, `approve(address, uint256)`, and `allowance(address, address)`. It also requires events: `Transfer` and `Approval`.

2. **What is the purpose of the approve and transferFrom pattern?**

   Answer: The approve/transferFrom pattern allows an account to authorize a third party (spender) to transfer tokens on its behalf. This is essential for applications like decentralized exchanges, where the exchange needs to move tokens from users' accounts to execute trades. It provides delegation without sharing private keys.

3. **How does the decimals function work in ERC-20 tokens?**

   Answer: The `decimals()` function returns the number of decimal places the token uses. For example, if decimals = 18, 1 token equals $10^18 base units. This allows for fractional token amounts. Most tokens use 18 decimals to match ETH'

4. **Why should you use OpenZeppelin for ERC-20 implementation?**

   Answer: OpenZeppelin provides battle-tested, secure implementations of ERC-20 and other standards. Using them saves development time, reduces the risk of bugs, and ensures compliance with standards. They've been audited and widely used in production, making them the preferred choice for secure smart contract development.

13 EXPERIMENT 13: CREATE A SINGLE BLOCK

Objective: To write a Python class for a single block storing an index, timestamp, data, previous hash, and nonce, and calculate its SHA-256 hash.

Hardware & Software Tools Required

- Python 3.x
- `hashlib` library
- VS Code/PyCharm IDE

Theoretical Background

A block is the fundamental building unit of a blockchain, containing data and cryptographic links to previous blocks.
Block Components:

1. **Index:** Position in the blockchain
2. **Timestamp:** Creation time
3. **Data:** Transaction information
4. **Previous Hash:** Hash of the previous block
5. **Nonce:** Number used for mining
6. **Hash:** Current block's hash

Complete Source Code

```
"""
Lab 13: Create a Single Block
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

import hashlib
import time
import json
from typing import Any, Dict

class Block:
    """
    A single block in the blockchain
    """

    def __init__(self, index: int, data: Any, previous_hash: str, nonce:
    int = 0):
```

```
        """
        Initialize a block

        Args:
            index (int): Block index
            data (Any): Block data
            previous_hash (str): Hash of the previous block
            nonce (int): Nonce value for mining
        """
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.calculate_hash()

    def calculate_hash(self) -> str:
        """
        Calculate the SHA-256 hash of the block

        Returns:
            str: Block hash in hexadecimal
        """
        # Combine all block data into a single string
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce
        }, sort_keys=True)

        # Calculate SHA-256 hash
        return hashlib.sha256(block_string.encode('utf-8')).hexdigest()

    def mine_block(self, difficulty: int) -> None:
        """
        Mine the block by finding a hash with leading zeros

        Args:
            difficulty (int): Number of leading zeros required
        """
        target = '0' * difficulty
        print(f"Mining block {self.index}...")

        while self.hash[:difficulty] != target:
            self.nonce += 1
            self.hash = self.calculate_hash()

        print(f"Block mined! Hash: {self.hash}")
        print(f"Nonce: {self.nonce}")

    def to_dict(self) -> Dict[str, Any]:
        """
        Convert block to dictionary

        Returns:
            Dict: Block data as dictionary
```

```
        """
        return {
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce,
            "hash": self.hash
        }

    def __str__(self) -> str:
        """String representation of the block"""
        return json.dumps(self.to_dict(), indent=2)

    def is_valid(self) -> bool:
        """
        Validate the block hash

        Returns:
            bool: True if valid, False otherwise
        """
        return self.hash == self.calculate_hash()

def main():
    """Main function to demonstrate block creation"""

    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 13")
    print("Create a Single Block")
    print("=" * 80)

    # 1. Create a genesis block
    print("\n[1] CREATING GENESIS BLOCK")
    print("-" * 60)

    genesis_block = Block(0, "Genesis Block", "0")
    print(f"Genesis Block:\n{genesis_block}")
    print(f"Is Valid: {genesis_block.is_valid()}")

    # 2. Create a block with data
    print("\n[2] CREATING A BLOCK WITH DATA")
    print("-" * 60)

    block_data = {
        "sender": "Alice",
        "receiver": "Bob",
        "amount": 100,
        "currency": "BTC"
    }

    block = Block(1, block_data, genesis_block.hash)
    print(f"Block:\n{block}")
    print(f"Is Valid: {block.is_valid()}")

    # 3. Modify block data and observe hash change
    print("\n[3] DATA TAMPERING DETECTION")
    print("-" * 60)

```

```
    original_hash = block.hash
    print(f"Original Hash: {original_hash}")

    # Tamper with the data
    block.data["amount"] = 1000
    new_hash = block.calculate_hash()
    print(f"Modified Hash: {new_hash}")
    print(f"Hash Changed: {original_hash != new_hash}")
    print(f"Is Valid: {block.is_valid()}")

    # 4. Mining demonstration
    print("\n[4] MINING DEMONSTRATION")
    print("-" * 60)

    mining_block = Block(2, "Block with mining", block.hash)
    print(f"Before Mining:\nNonce: {mining_block.nonce}")
    print(f"Hash: {mining_block.hash}")

    # Mine with difficulty 2
    mining_block.mine_block(2)
    print(f"\nAfter Mining:\nNonce: {mining_block.nonce}")
    print(f"Hash: {mining_block.hash}")
    print(f"Hash starts with: {mining_block.hash[:2]}")

    # 5. Block properties
    print("\n[5] BLOCK PROPERTIES")
    print("-" * 60)

    for attr in ["index", "timestamp", "data", "previous_hash", "nonce",
    "hash"]:
        print(f"{attr}: {getattr(block, attr, 'N/A')}")

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 28: Single Block Implementation

Viva-Voce Questions

1. **What are the essential components of a block in a blockchain?**

   Answer: The essential components include: (1) Index - position in the chain; (2) Timestamp - creation time; (3) Data - transaction information; (4) Previous Hash - cryptographic link to previous block; (5) Nonce - number used for mining; (6) Hash - the block's own hash calculated from all components.

2. **Why is the previous hash important in a blockchain?**

   Answer: The previous hash creates a cryptographic link between blocks, making the chain tamper-evident. If any block's data is modified, its hash changes, breaking the link and invalidating all subsequent blocks. This ensures the integrity of the entire blockchain and makes it computationally infeasible to modify historical data without detection.

3. **What is the purpose of the nonce in a block?**

   Answer: The nonce (number used once) is a variable that miners change to find a valid hash meeting the difficulty requirement. By incrementing the nonce and recalculating the hash, miners search for a hash with a specified number of leading zeros. The nonce ensures that mining is a proof-of-work process, requiring computational effort to create a valid block.

4. **How does the block hash ensure data integrity?**

   Answer: The block hash is calculated from all block data. Any change to the block data, even a single character, results in a completely different hash. This makes the block tamper-evident. The hash also serves as a unique identifier for the block and is used to link blocks together, ensuring the integrity of the entire blockchain.

14 EXPERIMENT 14: CREATE A MERKLE TREE

Objective: To write a Python program that takes a list of transactions, combines their hashes in pairs, and finds the single top hash (Merkle Root).

Hardware & Software Tools Required

- Python 3.x
- `hashlib` library
- VS Code/PyCharm IDE

Theoretical Background

A Merkle Tree is a binary tree structure used to efficiently verify the integrity of large datasets.

Key Concepts:

1. **Leaf Nodes:** Hashes of individual transactions
2. **Internal Nodes:** Hashes of child node pairs
3. **Merkle Root:** Single hash representing the entire tree
4. **Merkle Proof:** Path to verify a transaction's inclusion

Complete Source Code

```
"""
Lab 14: Create a Merkle Tree
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

import hashlib
import json
from typing import List, Tuple, Optional
import math

class MerkleTree:
    """
    Implementation of a binary Merkle Tree
    """

    def __init__(self, transactions: List[str]):
        """
        Initialize Merkle Tree from transactions

        Args:
            transactions (List[str]): List of transaction data
```

```
        """
        self.transactions = transactions
        self.leaves = self._hash_transactions(transactions)
        self.tree = self._build_tree(self.leaves)
        self.root = self.tree[-1][0] if self.tree else None

    def _hash_transactions(self, transactions: List[str]) -> List[str]:
        """
        Hash each transaction to create leaves

        Args:
            transactions (List[str]): Transaction data

        Returns:
            List[str]: Hashed leaves
        """
        return [self._hash_data(tx) for tx in transactions]

    def _hash_data(self, data: str) -> str:
        """
        Hash data using SHA-256

        Args:
            data (str): Data to hash

        Returns:
            str: SHA-256 hash
        """
        return hashlib.sha256(data.encode('utf-8')).hexdigest()

    def _hash_pair(self, left: str, right: str) -> str:
        """
        Hash a pair of nodes

        Args:
            left (str): Left node hash
            right (str): Right node hash

        Returns:
            str: Combined hash
        """
        combined = left + right
        return self._hash_data(combined)

    def _build_tree(self, leaves: List[str]) -> List[List[str]]:
        """
        Build the Merkle Tree

        Args:
            leaves (List[str]): Leaf hashes

        Returns:
            List[List[str]]: Tree levels (bottom to top)
        """
        if not leaves:
            return []

        # Ensure even number of leaves (duplicate last if odd)
```

```
        if len(leaves) % 2 == 1:
            leaves.append(leaves[-1])

        tree = [leaves]
        current_level = leaves

        while len(current_level) > 1:
            next_level = []

            # Pair up nodes
            for i in range(0, len(current_level), 2):
                left = current_level[i]
                right = current_level[i + 1] if i + 1 < len(
    current_level) else current_level[i]
                parent = self._hash_pair(left, right)
                next_level.append(parent)

            # Ensure even number of nodes
            if len(next_level) % 2 == 1 and len(next_level) > 1:
                next_level.append(next_level[-1])

            tree.append(next_level)
            current_level = next_level

        return tree

    def get_root(self) -> Optional[str]:
        """
        Get the Merkle Root

        Returns:
            Optional[str]: Merkle Root hash
        """
        return self.root

    def get_tree_levels(self) -> List[List[str]]:
        """
        Get all tree levels

        Returns:
            List[List[str]]: Complete tree structure
        """
        return self.tree

    def get_merkle_proof(self, transaction_index: int) -> List[Tuple[str
    , str]]:
        """
        Generate Merkle proof for a transaction

        Args:
            transaction_index (int): Index of the transaction

        Returns:
            List[Tuple[str, str]]: List of (hash, direction) pairs
            Direction: 'left' or 'right'
        """
        if transaction_index >= len(self.transactions):
            raise ValueError("Transaction index out of range")
```

```
138
139         if len(self.tree) == 0:
140             return []
141
142         proof = []
143         current_index = transaction_index
144
145         # Traverse from leaves to root
146         for level in range(len(self.tree) - 1):
147             nodes = self.tree[level]
148
149             # Determine if current node is left or right child
150             if current_index % 2 == 0:
151                 # Left child - get right sibling
152                 if current_index + 1 < len(nodes):
153                     sibling = nodes[current_index + 1]
154                     direction = 'right'
155                 else:
156                     sibling = nodes[current_index]  # If no right
    sibling, duplicate current
157                     direction = 'right'
158             else:
159                 # Right child - get left sibling
160                 sibling = nodes[current_index - 1]
161                 direction = 'left'
162
163             proof.append((sibling, direction))
164
165             # Move to parent level
166             current_index = current_index // 2
167
168         return proof
169
170     def verify_proof(self, transaction: str, proof: List[Tuple[str, str
    ]]) -> bool:
171         """
172         Verify a transaction using a Merkle proof
173
174         Args:
175             transaction (str): Transaction data to verify
176             proof (List[Tuple[str, str]]): Merkle proof
177
178         Returns:
179             bool: True if valid
180         """
181         current_hash = self._hash_data(transaction)
182
183         for sibling_hash, direction in proof:
184             if direction == 'left':
185                 current_hash = self._hash_pair(sibling_hash,
    current_hash)
186             else:  # 'right'
187                 current_hash = self._hash_pair(current_hash,
    sibling_hash)
188
189         return current_hash == self.root
190
191     def print_tree(self) -> None:
```

```
        """Pretty print the Merkle Tree"""
        print("\nMerkle Tree Structure:")
        for level_idx, level in enumerate(self.tree):
            indent = "  " * (len(self.tree) - level_idx)
            print(f"{indent}Level {level_idx}:")
            for i, node in enumerate(level):
                print(f"{indent}  Node {i}: {node[:16]}...")

def main():
    """Main function to demonstrate Merkle Tree"""

    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 14")
    print("Create a Merkle Tree")
    print("=" * 80)

    # 1. Create transactions
    print("\n[1] CREATING TRANSACTIONS")
    print("-" * 60)

    transactions = [
        "Alice -> Bob: 10 BTC",
        "Bob -> Carol: 5 BTC",
        "Carol -> Dave: 8 BTC",
        "Dave -> Alice: 3 BTC",
        "Alice -> Eve: 2 BTC",
        "Eve -> Carol: 4 BTC"
    ]

    for i, tx in enumerate(transactions):
        print(f"Transaction {i}: {tx}")

    # 2. Build Merkle Tree
    print("\n[2] BUILDING MERKLE TREE")
    print("-" * 60)

    merkle_tree = MerkleTree(transactions)
    tree_levels = merkle_tree.get_tree_levels()

    print(f"Number of Transactions: {len(transactions)}")
    print(f"Number of Tree Levels: {len(tree_levels)}")
    print(f"Merkle Root: {merkle_tree.get_root()}")

    # 3. Display Tree Structure
    print("\n[3] TREE STRUCTURE")
    print("-" * 60)
    merkle_tree.print_tree()

    # 4. Generate and Verify Proof
    print("\n[4] MERKLE PROOF GENERATION AND VERIFICATION")
    print("-" * 60)

    tx_index = 2  # Transaction "Carol -> Dave: 8 BTC"
    tx_data = transactions[tx_index]

    print(f"Transaction: {tx_data}")
    print(f"Transaction Index: {tx_index}")

```

```
    proof = merkle_tree.get_merkle_proof(tx_index)
    print("\nMerkle Proof:")
    for i, (hash_val, direction) in enumerate(proof):
        print(f"  Step {i}: {direction} -> {hash_val[:16]}...")

    # Verify the proof
    is_valid = merkle_tree.verify_proof(tx_data, proof)
    print(f"\nProof Verification: {'    VALID' if is_valid else '
    INVALID'}")

    # 5. Tampering Detection
    print("\n[5] TAMPERING DETECTION")
    print("-" * 60)

    # Tamper with a transaction
    tampered_data = "Carol -> Dave: 100 BTC"
    tampered_proof = merkle_tree.get_merkle_proof(tx_index)

    print(f"Original Data: {tx_data}")
    print(f"Tampered Data: {tampered_data}")

    original_valid = merkle_tree.verify_proof(tx_data, proof)
    tampered_valid = merkle_tree.verify_proof(tampered_data,
    tampered_proof)

    print(f"Original Proof Valid: {original_valid}")
    print(f"Tampered Proof Valid: {tampered_valid}")
    print(f"Tampering Detected: {'    YES' if not tampered_valid else '
       NO'}")

    # 6. Performance Test
    print("\n[6] PERFORMANCE TEST")
    print("-" * 60)

    for size in [10, 100, 1000]:
        txns = [f"Transaction {i}" for i in range(size)]
        tree = MerkleTree(txns)
        print(f"Transactions: {size} -> Tree Levels: {len(tree.
    get_tree_levels())}, Root: {tree.get_root()[:16]}...")

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 29: Merkle Tree Implementation

Viva-Voce Questions

1. **What is a Merkle Tree and why is it used in blockchain?**

   Answer: A Merkle Tree is a binary tree structure where each leaf is a hash of transaction data, and each internal node is the hash of its two children. It's used in blockchain to efficiently verify transaction inclusion without downloading the entire blockchain. It allows nodes to quickly verify that a transaction is part of a block

using only the Merkle Root and a short proof.

2. **How does the Merkle Root provide data integrity?**

   Answer: The Merkle Root is a single hash that represents all transactions in a block. Any change to any transaction will change its hash, propagate up the tree, and ultimately change the Merkle Root. This makes the block tamper-evident and allows for efficient verification of transaction inclusion.

3. **What is a Merkle proof and how does it work?**

   Answer: A Merkle proof is a set of hashes that allows a verifier to confirm that a transaction is included in a block. It consists of the sibling hashes along the path from the transaction leaf to the root. By hashing the transaction with these siblings in the correct order, the verifier can compute the Merkle Root and compare it with the known root.

4. **How does a Merkle Tree handle odd numbers of transactions?**

   Answer: When there's an odd number of transactions, the last transaction is duplicated (paired with itself) to create an even number of leaves. This ensures that all levels of the tree have an even number of nodes. This technique maintains the binary tree structure and is commonly used in practice.

15 EXPERIMENT 15: BUILD A COMPLETE BLOCKCHAIN

Objective: To write a Python program that links multiple blocks together in a chain using previous hashes and checks if any block was tampered with.

Hardware & Software Tools Required

- Python 3.x
- `hashlib` library
- VS Code/PyCharm IDE

Theoretical Background

A blockchain is a distributed ledger of immutable, sequentially linked blocks. Each block contains a hash of the previous block, creating an unbreakable chain.

Complete Source Code

```
"""
Lab 15: Build a Complete Blockchain
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

import hashlib
import time
import json
from typing import List, Dict, Any, Optional

class Block:
    """A block in the blockchain"""

    def __init__(self, index: int, data: Any, previous_hash: str, nonce:
    int = 0):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = nonce
        self.hash = self.calculate_hash()

    def calculate_hash(self) -> str:
        block_string = json.dumps({
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash,
```

```
            "nonce": self.nonce
        }, sort_keys=True)
        return hashlib.sha256(block_string.encode('utf-8')).hexdigest()

    def to_dict(self) -> Dict:
        return {
            "index": self.index,
            "timestamp": self.timestamp,
            "data": self.data,
            "previous_hash": self.previous_hash,
            "nonce": self.nonce,
            "hash": self.hash
        }

class Blockchain:
    """Complete blockchain implementation"""

    def __init__(self):
        self.chain: List[Block] = []
        self.difficulty = 2
        self.create_genesis_block()

    def create_genesis_block(self) -> None:
        """Create the first block in the chain"""
        genesis_block = Block(0, "Genesis Block", "0")
        genesis_block.hash = genesis_block.calculate_hash()
        self.chain.append(genesis_block)

    def get_latest_block(self) -> Block:
        """Get the most recent block in the chain"""
        return self.chain[-1]

    def mine_block(self, data: Any) -> None:
        """Mine and add a new block to the chain"""
        latest_block = self.get_latest_block()
        new_block = Block(
            index=latest_block.index + 1,
            data=data,
            previous_hash=latest_block.hash
        )

        # Mine the block
        print(f"Mining block {new_block.index}...")
        target = '0' * self.difficulty

        while new_block.hash[:self.difficulty] != target:
            new_block.nonce += 1
            new_block.hash = new_block.calculate_hash()

        print(f"Block mined! Hash: {new_block.hash}")
        print(f"Nonce: {new_block.nonce}")

        self.chain.append(new_block)

    def is_chain_valid(self) -> bool:
        """Verify the integrity of the entire blockchain"""
        for i in range(1, len(self.chain)):
            current_block = self.chain[i]
```

```
            previous_block = self.chain[i-1]

            # Verify current block's hash
            if current_block.hash != current_block.calculate_hash():
                print(f"Block {i} has invalid hash!")
                return False

            # Verify link to previous block
            if current_block.previous_hash != previous_block.hash:
                print(f"Block {i} has invalid previous hash!")
                return False

            # Verify proof-of-work
            if current_block.hash[:self.difficulty] != '0' * self.difficulty:
                print(f"Block {i} has invalid proof-of-work!")
                return False

        return True

    def tamper_with_block(self, index: int, new_data: Any) -> None:
        """Tamper with a block (for demonstration purposes)"""
        if 0 <= index < len(self.chain):
            self.chain[index].data = new_data
            self.chain[index].hash = self.chain[index].calculate_hash()
            print(f"Block {index} tampered with!")

    def get_chain_data(self) -> List[Dict]:
        """Get the entire blockchain as a list of dictionaries"""
        return [block.to_dict() for block in self.chain]

    def print_chain(self) -> None:
        """Print the entire blockchain"""
        print("\n" + "=" * 80)
        print("BLOCKCHAIN")
        print("=" * 80)

        for block in self.chain:
            print(f"\nBlock {block.index}:")
            print(f"  Timestamp: {block.timestamp}")
            print(f"  Data: {block.data}")
            print(f"  Previous Hash: {block.previous_hash[:16]}...")
            print(f"  Hash: {block.hash[:16]}...")
            print(f"  Nonce: {block.nonce}")

def main():
    """Main function to demonstrate complete blockchain"""

    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 15")
    print("Build a Complete Blockchain")
    print("=" * 80)

    # 1. Create blockchain
    print("\n[1] CREATING BLOCKCHAIN")
    print("-" * 60)

    blockchain = Blockchain()
```

```
    print("Blockchain created with genesis block!")
    blockchain.print_chain()

    # 2. Add blocks with transactions
    print("\n[2] ADDING BLOCKS")
    print("-" * 60)

    transactions = [
        {"sender": "Alice", "receiver": "Bob", "amount": 10},
        {"sender": "Bob", "receiver": "Carol", "amount": 5},
        {"sender": "Carol", "receiver": "Dave", "amount": 8},
        {"sender": "Dave", "receiver": "Alice", "amount": 3}
    ]

    for tx in transactions:
        blockchain.mine_block(tx)

    blockchain.print_chain()

    # 3. Verify chain
    print("\n[3] VERIFYING CHAIN INTEGRITY")
    print("-" * 60)

    is_valid = blockchain.is_chain_valid()
    print(f"Chain valid: {'    YES' if is_valid else '    NO'}")

    # 4. Tampering demonstration
    print("\n[4] TAMPERING DETECTION")
    print("-" * 60)

    print("Original block 1 data:", blockchain.chain[1].data)

    # Tamper with block 1
    blockchain.tamper_with_block(1, {"sender": "Eve", "receiver": "
    Mallory", "amount": 1000})
    print("After tampering:")
    print(f"Block 1 data: {blockchain.chain[1].data}")

    # Check validity after tampering
    is_valid = blockchain.is_chain_valid()
    print(f"Chain valid after tampering: {'    NO' if not is_valid else
    '       WARNING'}")

    # 5. Chain statistics
    print("\n[5] CHAIN STATISTICS")
    print("-" * 60)

    print(f"Total blocks: {len(blockchain.chain)}")
    print(f"Difficulty: {blockchain.difficulty}")

    total_time = 0
    for i in range(1, len(blockchain.chain)):
        block = blockchain.chain[i]
        prev_block = blockchain.chain[i-1]
        time_diff = block.timestamp - prev_block.timestamp
        total_time += time_diff

```

```
    avg_time = total_time / (len(blockchain.chain) - 1) if len(
    blockchain.chain) > 1 else 0
    print(f"Average block time: {avg_time:.2f} seconds")

    # 6. Export chain data
    print("\n[6] CHAIN DATA EXPORT")
    print("-" * 60)

    chain_data = blockchain.get_chain_data()
    print(json.dumps(chain_data[0], indent=2))

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 30: Complete Blockchain Implementation

Viva-Voce Questions

1. **How does the blockchain ensure immutability?**

   Answer: The blockchain ensures immutability through cryptographic linking of blocks. Each block contains the hash of the previous block, creating a chain. Any change to a block's data changes its hash, which breaks the link to the next block. Recalculating all subsequent hashes would require enormous computational power, making historical data effectively immutable.

2. **What is the role of the previous hash in the blockchain?**

   Answer: The previous hash creates the cryptographic link between blocks. It ensures that each block references the exact previous block. This linking mechanism makes it possible to detect tampering because modifying any block requires updating all subsequent blocks' previous hashes, which is computationally infeasible.

3. **How does the chain validation algorithm work?**

   Answer: Chain validation verifies: (1) each block's hash matches its computed hash; (2) each block's previous hash matches the actual previous block's hash; (3) each block meets the proof-of-work difficulty requirement. This ensures the entire chain is valid and hasn't been tampered with.

4. **What are the limitations of this implementation compared to real blockchains?**

   Answer: This implementation is simplified for educational purposes. Real blockchains have: (1) decentralized consensus mechanisms; (2) peer-to-peer networking for block propagation; (3) advanced merkle trees for transaction verification; (4) smart contracts and state management; (5) economic incentives for mining; (6) more sophisticated validation and error handling.

16 EXPERIMENT 16: MINING WITH PROOF OF WORK

Objective: To write a Python program that mines a block by changing the nonce until the hash starts with a given number of zeros (e.g., 0000).

Hardware & Software Tools Required

- Python 3.x
- `hashlib` library
- VS Code/PyCharm IDE

Theoretical Background

Proof of Work (PoW) is a consensus mechanism that requires computational effort to create valid blocks, making attacks economically infeasible.

Complete Source Code

```
"""
Lab 16: Mining with Proof of Work
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

import hashlib
import time
import json
from typing import Dict, Tuple
import matplotlib.pyplot as plt

class ProofOfWork:
    """
    Implementation of Proof of Work mining algorithm
    """

    def __init__(self, difficulty: int = 4):
        """
        Initialize PoW miner

        Args:
            difficulty (int): Number of leading zeros required
        """
        self.difficulty = difficulty
        self.target = '0' * difficulty

    def mine_block(self, block_data: Dict) -> Tuple[int, str, float]:
        """
```

```
        Mine a block by finding a valid nonce

        Args:
            block_data (Dict): Block data to mine

        Returns:
            Tuple[int, str, float]: (nonce, hash, time_taken)
        """
        start_time = time.time()
        nonce = 0
        max_nonce = 10**8  # Prevent infinite loops

        while nonce < max_nonce:
            # Create block string with nonce
            block_string = json.dumps({
                'data': block_data,
                'nonce': nonce
            }, sort_keys=True)

            # Calculate hash
            block_hash = hashlib.sha256(block_string.encode()).hexdigest
    ()

            # Check if hash meets difficulty
            if block_hash[:self.difficulty] == self.target:
                end_time = time.time()
                return nonce, block_hash, end_time - start_time

            nonce += 1

        raise Exception(f"Could not mine block within {max_nonce} nonce
    limit")

    def verify_block(self, block_data: Dict, nonce: int, block_hash: str
    ) -> bool:
        """
        Verify a mined block

        Args:
            block_data (Dict): Block data
            nonce (int): Nonce value
            block_hash (str): Claimed hash

        Returns:
            bool: True if valid
        """
        block_string = json.dumps({
            'data': block_data,
            'nonce': nonce
        }, sort_keys=True)

        computed_hash = hashlib.sha256(block_string.encode()).hexdigest
    ()

        return (computed_hash == block_hash and
                computed_hash[:self.difficulty] == self.target)

def main():
```

```
    """Main function to demonstrate PoW mining"""

    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 16")
    print("Mining with Proof of Work")
    print("=" * 80)

    # 1. Basic mining
    print("\n[1] BASIC MINING")
    print("-" * 60)

    block_data = {
        "index": 1,
        "timestamp": time.time(),
        "transactions": [
            {"sender": "Alice", "receiver": "Bob", "amount": 10},
            {"sender": "Bob", "receiver": "Carol", "amount": 5}
        ],
        "previous_hash":
    "0000000000000000000000000000000000000000000000000000000000000000"
    }

    miner = ProofOfWork(4)  # Difficulty 4 (leading zeros)

    print(f"Mining with difficulty {miner.difficulty}")
    print(f"Target: {miner.target}")
    print(f"Block data: {json.dumps(block_data, indent=2)}")

    nonce, block_hash, time_taken = miner.mine_block(block_data)

    print(f"\  n      BLOCK MINED!")
    print(f"Nonce: {nonce}")
    print(f"Hash: {block_hash}")
    print(f"Time taken: {time_taken:.4f} seconds")
    print(f"Hash starts with: {block_hash[:miner.difficulty]}")

    # 2. Verify block
    print("\n[2] VERIFYING BLOCK")
    print("-" * 60)

    is_valid = miner.verify_block(block_data, nonce, block_hash)
    print(f"Block valid: {'    YES' if is_valid else '    NO'}")

    # 3. Different difficulties
    print("\n[3] COMPARING DIFFICULTIES")
    print("-" * 60)

    difficulties = [2, 3, 4, 5]
    results = []

    for diff in difficulties:
        miner_diff = ProofOfWork(diff)
        print(f"\nMining with difficulty {diff}...")
        start = time.time()

        try:
            nonce, block_hash, time_taken = miner_diff.mine_block(
    block_data)
```

```
            results.append({
                'difficulty': diff,
                'time': time_taken,
                'nonce': nonce,
                'hash': block_hash
            })
            print(f"  Nonce: {nonce}")
            print(f"  Time: {time_taken:.4f} seconds")
            print(f"  Hash: {block_hash[:16]}...")

        except Exception as e:
            print(f"  Failed: {e}")

    # 4. Performance analysis
    print("\n[4] PERFORMANCE ANALYSIS")
    print("-" * 60)

    if results:
        for r in results:
            print(f"Difficulty {r['difficulty']}: {r['time']:.4f}s, Nonce: {r['nonce']}")

        # Calculate average hashrate
        total_time = sum(r['time'] for r in results)
        total_nonce = sum(r['nonce'] for r in results)
        avg_hashrate = total_nonce / total_time if total_time > 0 else 0
        print(f"\nAverage hashrate: {avg_hashrate:.1f} hashes/second")

    # 5. Edge cases
    print("\n[5] EDGE CASES")
    print("-" * 60)

    # Test with different block data
    test_cases = [
        {"data": "small_data", "description": "Small data"},
        {"data": "x" * 1000, "description": "Large data"},
        {"data": {"key": "value", "nested": {"deep": "data"}}, "description": "Complex data"}
    ]

    for test in test_cases:
        print(f"\nTesting: {test['description']}")
        try:
            nonce, hash_val, time_taken = miner.mine_block(test['data'])
            print(f"  Success! Nonce: {nonce}, Time: {time_taken:.4f}s")
            print(f"  Hash: {hash_val[:20]}...")
        except Exception as e:
            print(f"  Failed: {e}")

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 31: Proof of Work Mining

Viva-Voce Questions

1. **What is Proof of Work and why is it important?**

   Answer: Proof of Work is a consensus mechanism requiring computational effort to create new blocks. It solves the Sybil attack problem by making it expensive to produce blocks. The difficulty ensures that creating valid blocks requires significant computation, making it economically infeasible for attackers to manipulate the chain.

2. **How does the difficulty affect mining time?**

   Answer: Higher difficulty means more leading zeros required in the hash, which exponentially increases the mining time. Each additional leading zero roughly doubles the expected number of hashes needed (about 16x increase per hex digit). The difficulty adjusts to maintain a consistent block time.

3. **What is the role of the nonce in mining?**

   Answer: The nonce is a variable that miners increment to find a valid hash. By changing the nonce, the resulting hash changes pseudo-randomly. Miners try billions of nonce values until they find one that produces a hash meeting the difficulty requirement.

4. **Why can't we simply change the difficulty value in a real blockchain?**

   Answer: In real blockchains, the difficulty is adjusted automatically based on the network's total hashrate. It's determined by consensus rules and can only be changed by protocol upgrades. This ensures network security and stability. Manual difficulty changes would break the consensus and could be exploited.

17 EXPERIMENT 17: CONSENSUS ALGORITHMS SIMULATION

Objective: To write a Python script that simulates how Proof of Work (mining race) and Proof of Stake (coin-weight selection) choose the next block creator.

Hardware & Software Tools Required

- Python 3.x
- `random` library
- VS Code/PyCharm IDE

Theoretical Background

Consensus algorithms determine who gets to create the next block in a blockchain network.

Complete Source Code

```
"""
Lab 17: Consensus Algorithms Simulation
Course: CSCSE104 - Blockchain & Applications
Author: [Student Name]
Date: [Current Date]
"""

import random
import time
import hashlib
from typing import List, Dict, Tuple
from collections import defaultdict
import math

class Node:
    """Represents a node in the network"""

    def __init__(self, node_id: int, stake: float = 0):
        self.node_id = node_id
        self.stake = stake
        self.hash_power = random.random()  # Random mining power
        self.blocks_mined = 0
        self.rewards = 0

    def __str__(self):
        return f"Node {self.node_id}: Stake={self.stake:.2f}, Power={
   self.hash_power:.2f}"

class ProofOfWork:
    """Proof of Work consensus simulation"""

```

```
    def __init__(self, nodes: List[Node], difficulty: int = 3):
        self.nodes = nodes
        self.difficulty = difficulty
        self.target = '0' * difficulty
        self.blocks_created = []

    def mine_round(self, round_num: int) -> Tuple[int, float]:
        """
        Simulate a mining round

        Returns:
            Tuple[int, float]: (Winner node ID, mining time)
        """
        start_time = time.time()

        # Each node tries to mine
        winners = []

        for node in self.nodes:
            # Simulate mining with hash power
            attempts = int(node.hash_power * 1000) + 1

            # Each attempt is a random hash
            for attempt in range(attempts):
                # Generate random nonce
                nonce = random.randint(0, 10**6)
                # Simulate hash calculation
                hash_val = hashlib.sha256(
                    f"{round_num}{node.node_id}{nonce}".encode()
                ).hexdigest()

                # Check if meets difficulty
                if hash_val[:self.difficulty] == self.target:
                    winners.append((node, hash_val, attempt))
                    break

        # Find the winner (first to find valid hash)
        if winners:
            winner, winner_hash, attempts = min(winners, key=lambda x: x
    [2])
            elapsed_time = time.time() - start_time

            # Update stats
            winner.blocks_mined += 1
            winner.rewards += 10  # Block reward
            self.blocks_created.append((round_num, winner.node_id))

            return winner.node_id, elapsed_time

        # No winner - rare but possible
        return -1, time.time() - start_time

class ProofOfStake:
    """Proof of Stake consensus simulation"""

    def __init__(self, nodes: List[Node]):
        self.nodes = nodes
        self.total_stake = sum(node.stake for node in nodes)
```

```
        self.blocks_created = []

    def select_validator(self) -> Node:
        """
        Select a validator based on stake weight

        Returns:
            Node: Selected validator
        """
        if self.total_stake == 0:
            # Equal chance if no stake
            return random.choice(self.nodes)

        # Weighted random selection
        choice = random.uniform(0, self.total_stake)
        current_sum = 0

        for node in self.nodes:
            current_sum += node.stake
            if current_sum >= choice:
                return node

        return self.nodes[-1]  # Fallback

    def create_block(self, round_num: int) -> Tuple[int, float]:
        """
        Create a block using PoS

        Returns:
            Tuple[int, float]: (Selected node ID, time taken)
        """
        start_time = time.time()

        # Select validator
        validator = self.select_validator()

        # Simulate block creation time
        time.sleep(random.uniform(0.01, 0.05))

        # Update stats
        validator.blocks_mined += 1
        validator.rewards += 10  # Block reward
        self.blocks_created.append((round_num, validator.node_id))

        return validator.node_id, time.time() - start_time

def main():
    """Main function to simulate consensus algorithms"""

    print("=" * 80)
    print("BLOCKCHAIN & APPLICATIONS LAB - EXPERIMENT 17")
    print("Consensus Algorithms Simulation")
    print("=" * 80)

    # 1. Create network
    print("\n[1] CREATING NETWORK")
    print("-" * 60)

```

```
    num_nodes = 10
    nodes = []

    for i in range(num_nodes):
        # Assign random stakes (between 0 and 100)
        stake = random.uniform(0, 100)
        nodes.append(Node(i, stake))

    print(f"Network created with {num_nodes} nodes")
    print("Node distribution:")
    for node in nodes[:5]:  # Show first 5
        print(f"  {node}")
    print(f"  ... and {num_nodes - 5} more nodes")

    # 2. Proof of Work Simulation
    print("\n[2] PROOF OF WORK SIMULATION")
    print("-" * 60)

    pow_consensus = ProofOfWork(nodes.copy(), difficulty=3)
    pow_winners = []
    pow_times = []

    print("Running PoW rounds...")
    for round_num in range(10):
        winner, time_taken = pow_consensus.mine_round(round_num)
        pow_winners.append(winner)
        pow_times.append(time_taken)
        if winner != -1:
            print(f"  Round {round_num}: Node {winner} won in {time_taken:.4f}s")
        else:
            print(f"  Round {round_num}: No winner found")

    # 3. Proof of Stake Simulation
    print("\n[3] PROOF OF STAKE SIMULATION")
    print("-" * 60)

    pos_consensus = ProofOfStake(nodes.copy())
    pos_winners = []
    pos_times = []

    print("Running PoS rounds...")
    for round_num in range(10):
        winner, time_taken = pos_consensus.create_block(round_num)
        pos_winners.append(winner)
        pos_times.append(time_taken)
        print(f"  Round {round_num}: Node {winner} selected in {time_taken:.4f}s")

    # 4. Comparison Analysis
    print("\n[4] CONSENSUS COMPARISON")
    print("-" * 60)

    # PoW Statistics
    valid_pow_winners = [w for w in pow_winners if w != -1]
    if valid_pow_winners:
        pow_avg_time = sum(pow_times) / len(pow_times)
        pow_unique_winners = len(set(valid_pow_winners))
```

```
        print(f"\nProof of Work Statistics:")
        print(f"  Total rounds: {len(pow_times)}")
        print(f"  Successful rounds: {len(valid_pow_winners)}")
        print(f"  Success rate: {len(valid_pow_winners)/len(pow_times)*100:.1f}%")
        print(f"  Average time per round: {pow_avg_time:.4f}s")
        print(f"  Unique winners: {pow_unique_winners}")

    # PoS Statistics
    pos_avg_time = sum(pos_times) / len(pos_times)
    pos_unique_winners = len(set(pos_winners))
    print(f"\nProof of Stake Statistics:")
    print(f"  Total rounds: {len(pos_times)}")
    print(f"  Success rate: 100%")
    print(f"  Average time per round: {pos_avg_time:.4f}s")
    print(f"  Unique winners: {pos_unique_winners}")

    # 5. Performance Evaluation
    print("\n[5] PERFORMANCE EVALUATION")
    print("-" * 60)

    # Vary number of nodes
    node_counts = [5, 10, 20, 50]

    print("Scaling test with different node counts:")
    for count in node_counts:
        test_nodes = [Node(i, random.uniform(0, 100)) for i in range(count)]

        # Test PoW
        pow_test = ProofOfWork(test_nodes, difficulty=2)
        start = time.time()
        for r in range(5):
            pow_test.mine_round(r)
        pow_time = time.time() - start

        # Test PoS
        pos_test = ProofOfStake(test_nodes)
        start = time.time()
        for r in range(5):
            pos_test.create_block(r)
        pos_time = time.time() - start

        print(f"\nNodes: {count}")
        print(f"  PoW time: {pow_time:.4f}s (avg: {pow_time/5:.4f}s/round)")
        print(f"  PoS time: {pos_time:.4f}s (avg: {pos_time/5:.4f}s/round)")

    # 6. Stake Distribution Analysis
    print("\n[6] STAKE DISTRIBUTION EFFECT")
    print("-" * 60)

    # Create nodes with different stake distributions
    distributions = [
        ("Equal", [100, 100, 100, 100, 100]),
        ("Skewed", [300, 100, 50, 30, 20]),
        ("Highly Skewed", [1000, 50, 30, 20, 10])
```

```
    ]

    for name, stakes in distributions:
        nodes_dist = [Node(i, stake) for i, stake in enumerate(stakes)]
        pos_dist = ProofOfStake(nodes_dist)

        print(f"\n{name} Distribution:")
        print(f"  Stakes: {stakes}")

        winners = []
        for r in range(20):
            winner, _ = pos_dist.create_block(r)
            winners.append(winner)

        # Count winner frequencies
        from collections import Counter
        freq = Counter(winners)
        total = len(winners)

        print(f"  Winner frequencies (20 rounds):")
        for node_id, count in sorted(freq.items()):
            percentage = (count / total) * 100
            print(f"    Node {node_id}: {count} ({percentage:.1f}%)")

    print("\n" + "=" * 80)
    print("LAB COMPLETED SUCCESSFULLY")
    print("=" * 80)

if __name__ == "__main__":
    main()
```

Listing 32: Consensus Algorithm Simulation

Viva-Voce Questions

1. **What are the main differences between PoW and PoS?**

   Answer: PoW requires computational work (mining) to create blocks and consumes significant energy. PoS selects validators based on their stake (cryptocurrency holdings), requiring no computation. PoW is more secure but slower and resource-intensive; PoS is faster, energy-efficient, but requires careful design to prevent "nothing at stake" attacks.

2. **How does the mining race work in PoW?**

   Answer: In PoW, all nodes (miners) compete simultaneously to find a valid hash. The first to find the solution wins the right to create the block and receives the block reward. The mining difficulty adjusts to maintain consistent block times regardless of network hashrate.

3. **What factors influence validator selection in PoS?**

   Answer: Validator selection is primarily based on the amount of stake (cryptocurrency held). Other factors include: age of stake (some implementations), randomness for fairness, and the validator's reputation (performance, uptime). The selection is weighted by stake amount.

4. **Which consensus algorithm is more suitable for what scenarios?**

   Answer: PoW is suitable for public, permissionless blockchains requiring high security (like Bitcoin) where energy consumption is acceptable. PoS is better for public blockchains with high transaction throughput needs (like Ethereum 2.0), enterprise applications, and scenarios where energy efficiency and speed are priorities.